Back-End APIs

HTTP, RESTful APIs, Postman, APIs with Node.js and Next.js, JWT Tokens

Svetlin Nakov, PhD

Co-founder @ SoftUni

SoftUni AI

https://ai.softuni.bg

Agenda

SoftUni AI

1. **Intro to Back-End Technologies**: client-server, APIs, databases, deployment
2. **HTTP Protocol**: requests, responses, methods, status codes; **URL format**
3. **RESTful APIs and Postman**: REST principles, testing with Postman, GitHub API
4. **Minimalistic TODO List API with Node.js**: build, run, test with Postman
5. **RESTful API Design Guidelines**: resource naming, methods, status codes, JSON
6. **Next.js Basics and APIs**: pages, API route handlers, implementing REST APIs
7. **TODO List API with Next.js**: CRUD endpoints (GET, POST, PATCH, DELETE)
8. **TODO List Client App**: view / add / edit / delete TODOs in Next.js
9. **Authentication & Authorization**: JWT tokens, Bearer auth, login flow
10. **TODO List API + Client with Auth**: register, login, protected endpoints
11. **Deploy Next.js App to Netlify**: publishing process, serverless limitations

Sli.do Code

AppsAI

Join at

slido.com

#AppsAI

Breaks

20:00 / 21:00

Intro to Back-End Technologies

Back-End, Client-Server, APIs, Databases, Deployment, Serverless

Client-Server Model

SoftUni AI

🖥️ Front-End (Client)

What users see and interact with

- HTML, CSS, JS, React, Vue, Angular, Bootstrap, …
- Runs in the **Web browser**

⚙️ Back-End (Server)

Business logic, data, storage, security

- Node.js, Python, Java, C#, Go, PHP, …
- Runs on a **server** (or serverless function)

🔄 Front-End ⇄ Back-End Communication

- Over **HTTP** (or **HTTPS**)
- Typically, through **APIs** (JSON)
- Flow: **Browser** ↔ **HTTP** ↔ **Server** ↔ **Database**

What is an API?

SoftUni AI

📡 API == Application Programming Interface

- A **contract**: how two programs talk to each other (set of rules)
- On the Web: client sends a **request** → server returns a **response**

🧩 Common API Styles

- **REST** (RESTful APIs) – resource-based, uses HTTP verbs
- **GraphQL** – single endpoint, query language for data
- **gRPC** / **WebSockets** / **SOAP** – less common in Web apps

👉 In this lesson: we focus on **RESTful APIs**

Databases and ORM Frameworks

SoftUni AI

🗄️ Databases

Store app data persistently

- **Relational**: PostgreSQL, MySQL, SQLite, SQL Server
- **Non-relational**: MongoDB, Redis, DynamoDB

🔗 ORM Frameworks

Object-Relational Mapping

- Map DB table rows to **objects** in the code
- Node.js ORMs: **Prisma**, **Drizzle**, **TypeORM**, **Sequelize**

👉 In this lesson: we keep data in a **simple JSON file**

No DB – focus on **HTTP**, **REST**, and **APIs** first

Deployment, Cloud, Containers

SoftUni AI

🌐 Where Does the Back-End Run?

- **Traditional servers**: VPS, dedicated hosts (DigitalOcean, Hetzner)
- **Cloud platforms**: AWS, Azure, Google Cloud
- **PaaS**: Netlify, Vercel, Railway, Render
- **Containers**: Docker + Kubernetes

🛠️ Two Common Hosting Styles

- **Server-based**: a Node.js process runs 24/7 (takes resources)
- **Serverless**: code runs on-demand, auto-scales, no server to manage

Intro to the HTTP Protocol

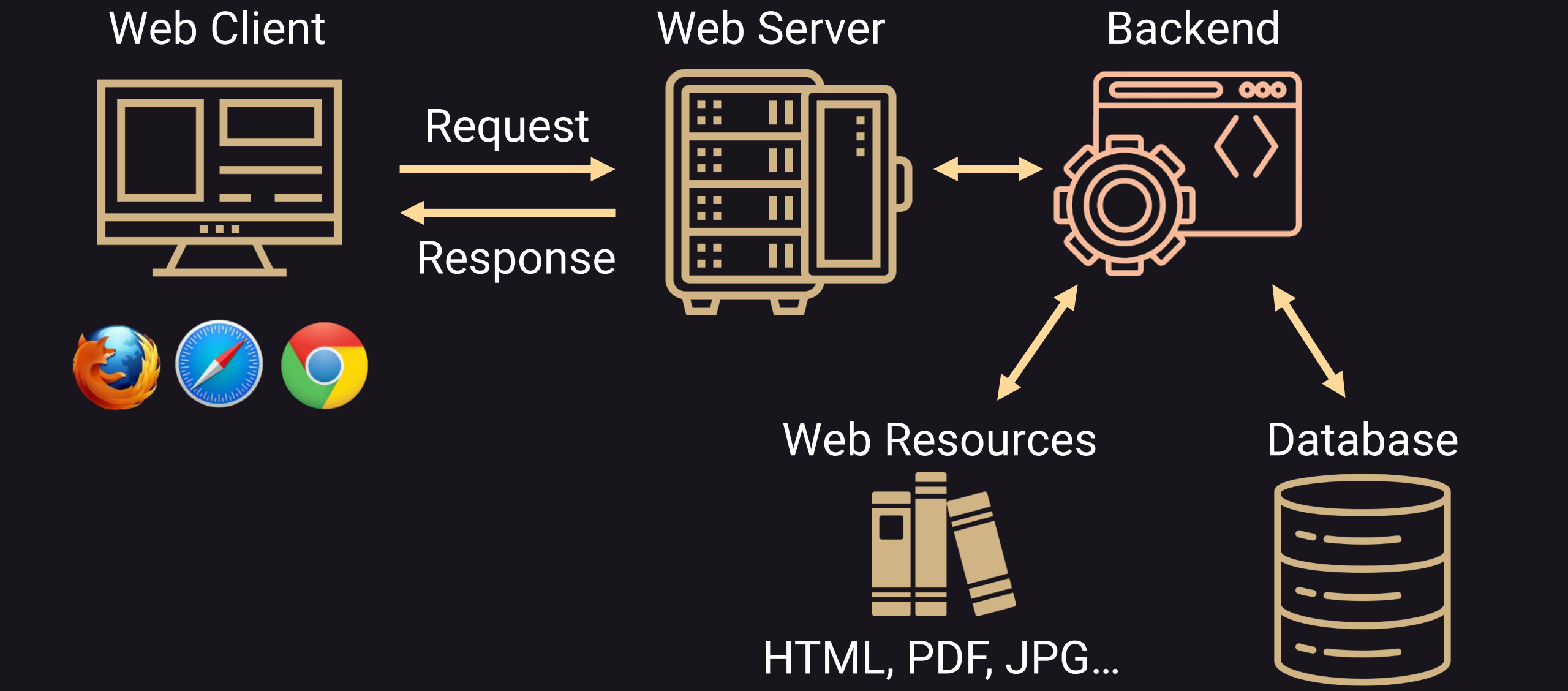
Requests, Responses, Methods, Status Codes

HTTP Basics

SoftUni AI

- **HTTP** (**H**yper**T**ext **T**ransfer **P**rotocol)
  - Text-based **client-server protocol** for the Internet
  - For transferring Web **resources**: HTML, images, styles, etc.
  - **Request-response** based, relies on URLs (like https://softuni.org)
  - **Stateless** (no memory between requests)

The Client-Server Model in Web Apps



HTTP Request Format

SoftUni AI

📥 HTTP Request Parts

- **Request line**
  - HTTP method + path + version
  - *e.g.* **GET /api/todos HTTP/1.1**
- **Headers**
  - Metadata as key-value pairs
  - **Host**, **Content-Type**, **Authorization**, **User Agent**
- **Body** (optional)
  - JSON, form data, file, …

Example – HTTP POST Request

```
POST /api/todos HTTP/1.1
Host: example.com
Content-Type: application/json
Authorization: Bearer eyJhbGciOi…

{ "title": "Learn JS and React",
"completed": false }
```

HTTP Request Methods

SoftUni AI

- **HTTP** defines **methods** to indicate the desired action to be performed on the identified resource

Method	Description
GET 	Retrieve a resource
POST 	Create / store a resource
PUT 	Update (replace) a resource
DELETE 	Delete (remove) a resource
PATCH 	Update resource partially (modify)
HEAD 	Retrieve the resource's headers

CRUD == the four main functions of persistent storage

Method
CONNECT
OPTIONS
TRACE

HTTP Response Format

SoftUni AI

HTTP Response Parts

- **Status line**
  - Version + status code + status text
  - *e.g.* **HTTP/1.1 201 Created**
- **Headers**
  - Metadata about the response
  - **Content-Type**, **Content-Length**, **Set-Cookie**
- **Body**
  - The returned **data** — HTML / JSON / image / script / …

Example – HTTP Response

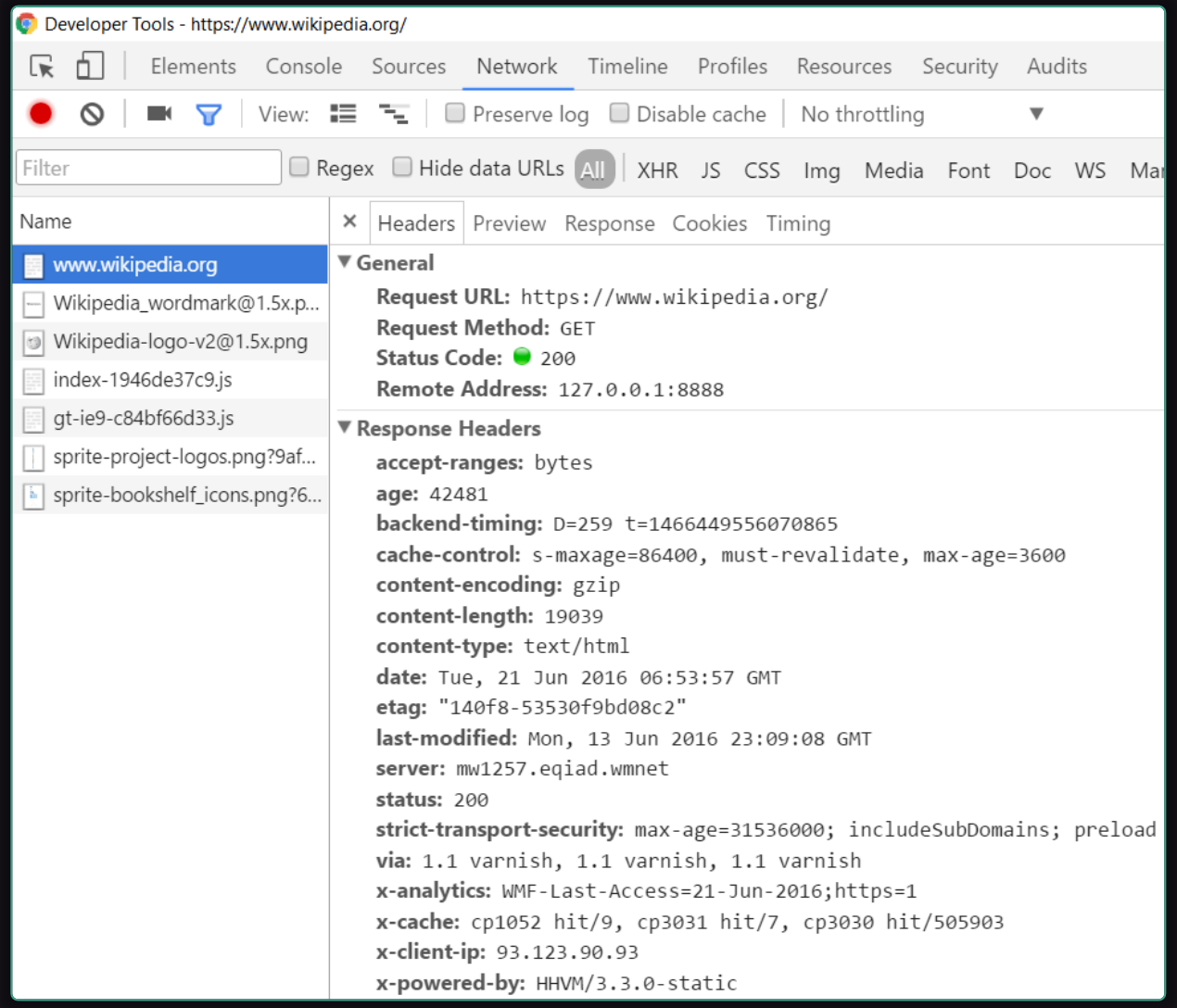
```
HTTP/1.1 201 Created
Content-Type:
application/json
Content-Length: 62
Set-Cookie: lang=en; Path=/

{ "id": 7, "title": "Learn JS
and React", "completed":
false }
```

HTTP Response Status Codes

SoftUni AI

Status Code	Action	Description
200	OK	Successfully retrieved resource
201	Created	A new resource was created
204	No Content	Request has nothing to return
301 / 302	Moved	Moved to another location (redirect)
400	Bad Request	Invalid request / syntax error
401 / 403	Unauthorized	Authentication failed / Access denied
404	Not Found	Invalid resource was requested
409	Conflict	Conflict was detected, e.g. duplicated email
500 / 503	Server Error	Internal server error / Service unavailable

Browser Dev Tools: Network Inspector



- **Chrome Dev Tools**
  - Press **[F12]** in Chrome
  - Open the [Network] tab
  - Inspect the HTTP traffic

Example: Fetch Existing URL (200 OK)

- Use the Web browser, **Postman**, or **curl** to send this **request**:

```
GET https://api.github.com/users/nakov
```

- The HTTP server **responds** with the requested resource:

HTTP response (200 OK)

```
HTTP/1.1 200 OK
Content-Type: application/json

{ "login": "nakov", "id": 1165408, "public_repos": 57, … }
```

Example: Fetch Non-Existing URL (404)

- Request a resource that **does not exist**:

```
https://api.github.com/users/no-such-user72739
```

- The HTTP server replies with **404 Not Found**:

HTTP response (404 Not Found)

```
HTTP/1.1 404 Not Found
Content-Type: application/json

{ "message": "Not Found", "documentation_url": "…" }
```

URLs and URL Format

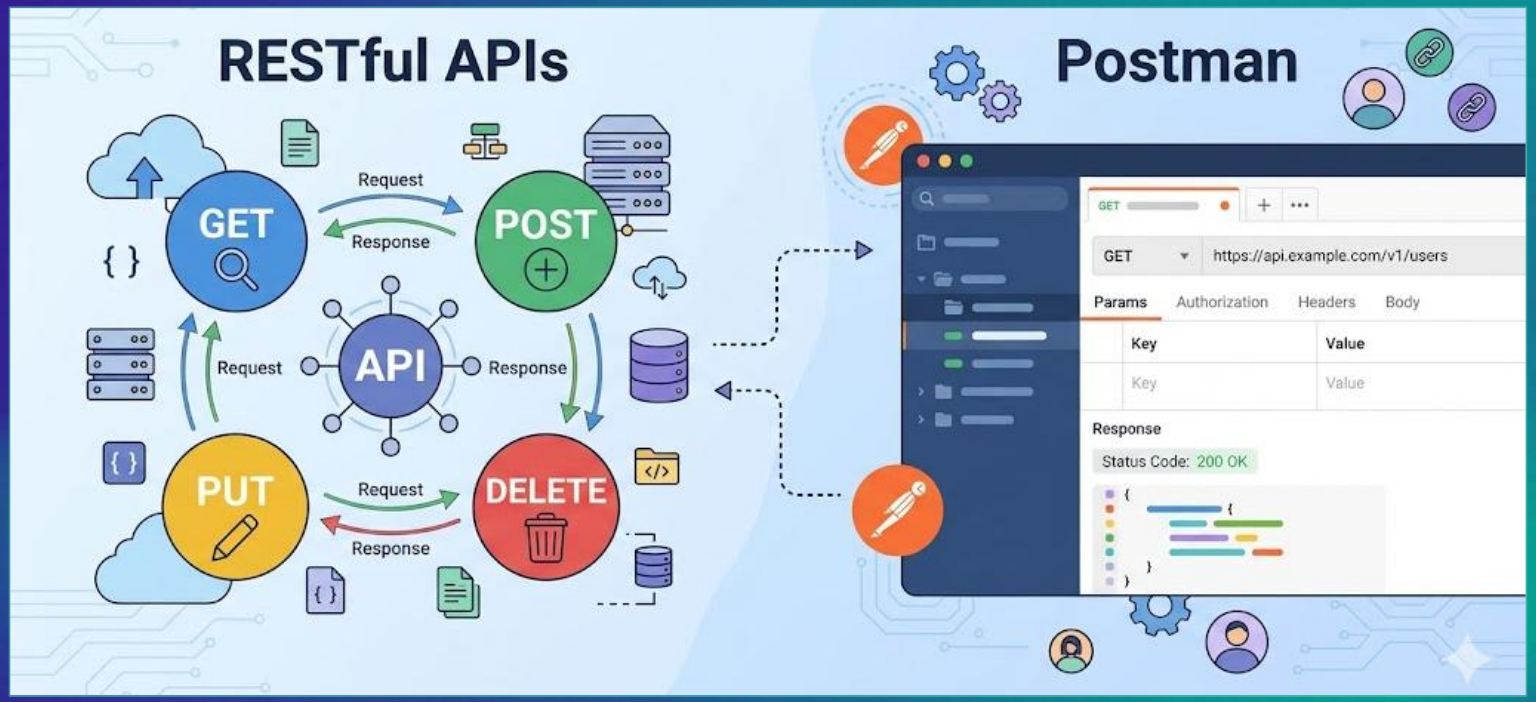
Protocol, Host, Path, Query String

Uniform Resource Locator (URL)

- Network **protocol** (**http**, **ftp**, **https**, …) – HTTP in most cases
- **Host** or **IP address**, e.g. **gmail.com**, **127.0.0.1**, **websrv**
- **Port** (the default port is **80**) – integer in the range [0…65535]
- **Path**, e.g. **/news**, **/products/7**, **/scripts/index.php**
- **Query string**, e.g. **?id=27&lang=en**
- **Fragment**, e.g. **#slides** – a HTML section in the page

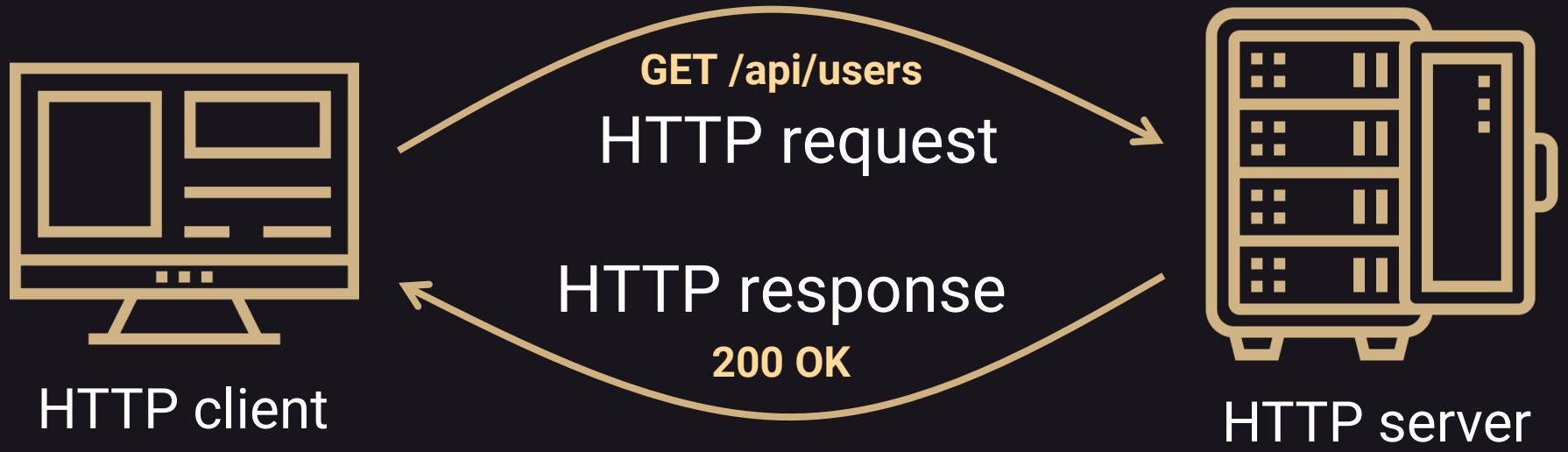
RESTful APIs and Postman

REST Principles, Playing with RESTful APIs with Postman, Example: GitHub API



RESTful APIs

- **RESTful APIs** are HTTP-based backend APIs (Web services)
  - HTTP methods **GET**, **POST**, **PATCH** and **DELETE** implement the **CRUD** operations (retrieve, create, modify and delete data)



RESTful APIs: Principles

SoftUni AI

- A **RESTful API** exposes server resources via **URLs** and **HTTP**
  - Example of server resource: **/api/users/42**
- **HTTP method** == the **action** (GET → read, POST → create, PUT/PATCH → update, DELETE → remove)
- Data is typically **JSON** (set Content-Type: application/json)
- **Status codes** tell what happened (200 → OK, 201 → Created, 401 → Unauthorized, 404 → Not Found, 500 → Server Error)
- **Stateless** – no sessions; send auth token with every request
- **Endpoints** are **nouns** (not verbs): **/api/users** (not **/getUsers**)

Example – GitHub API – List Repos

SoftUni AI

List Public GitHub Repos

For any GitHub user — **no auth needed**

- **Response**: JSON array of repo objects

Try in the browser:

- Just open the URL
- HTTP GET can be sent directly

Example: List Repos

```
GET https://api.github.com
/users/nakov/repos
```

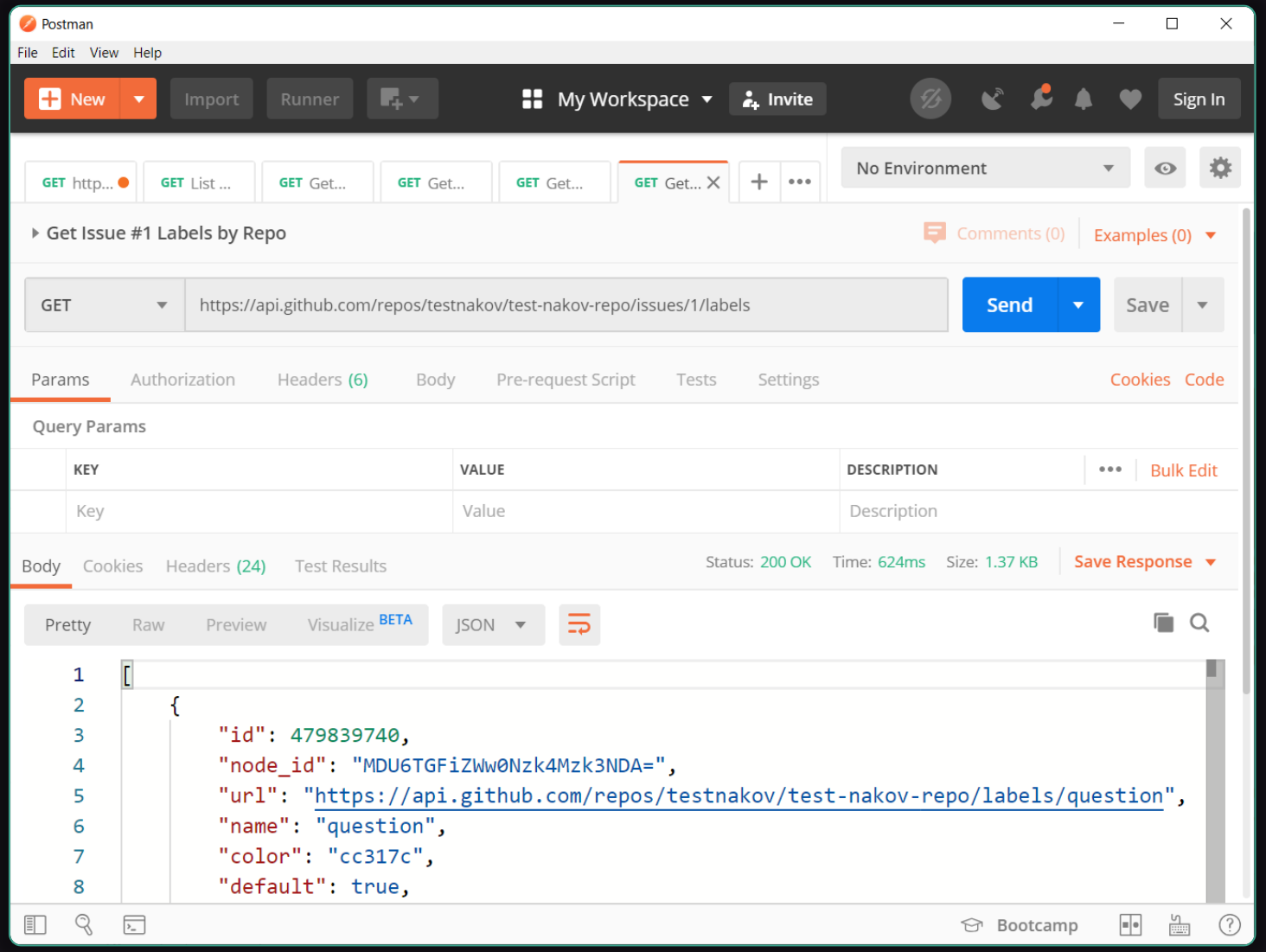
```
// Response (truncated)
[
  { "name": "softuni-ai",
    "stargazers_count": 42 },
  …
]
```

Postman: HTTP Testing Tool for Devs

SoftUni AI

Postman

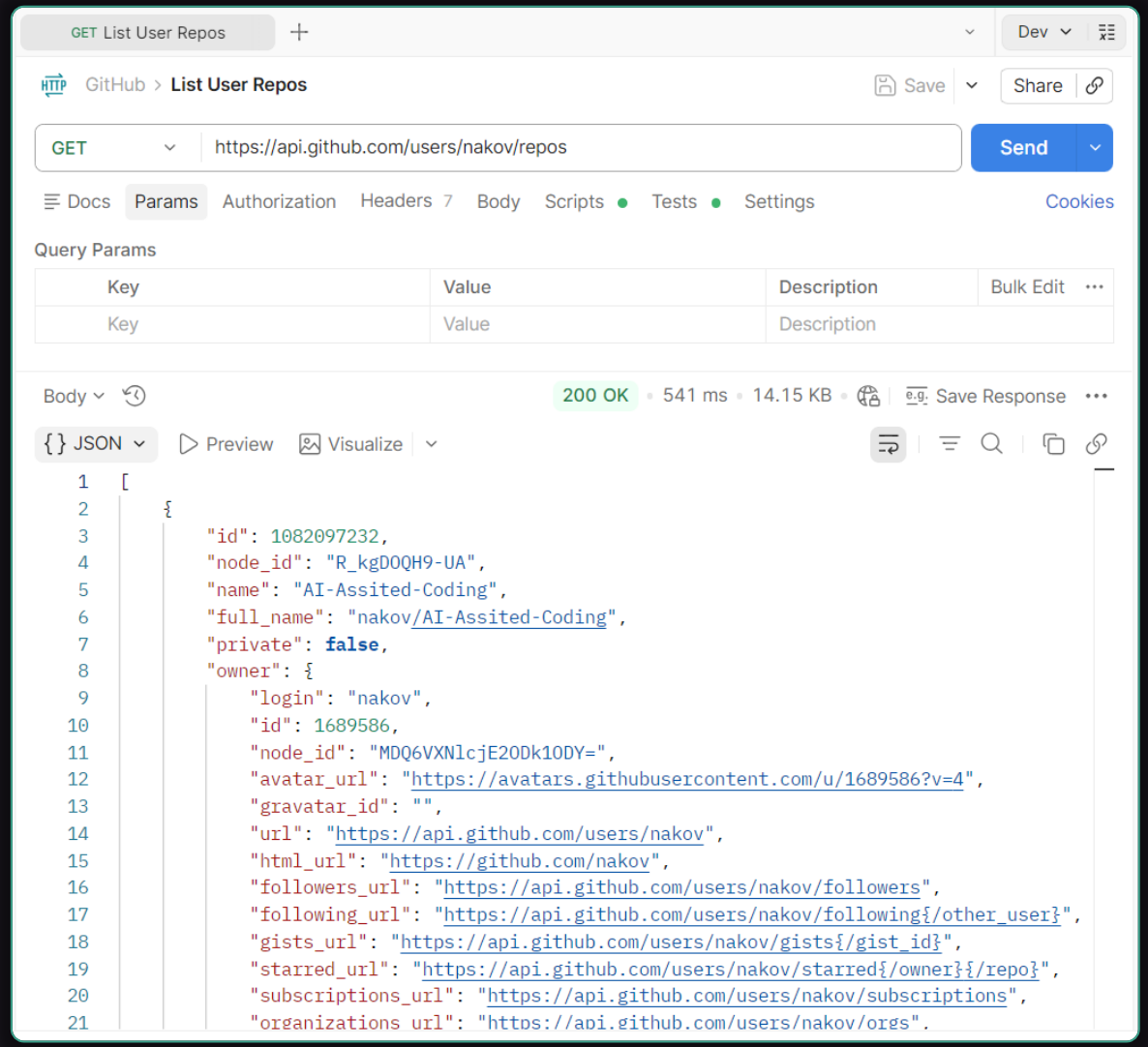
- HTTP client tool for developers
- Compose and send **HTTP requests**
- Alternatives
  - Insomnia, SoapUI, Hoppscotch



Example – GitHub API with Postman

SoftUni AI

- Try the "List GitHub Repos" request in **Postman**:
  - Method: **GET**
  - URL: **https://api.github.com/users/{user}/repos**
  - Click [Send] → inspect the HTTP response



Example — Minimalistic HTML + JS App

SoftUni AI

- **AI prompt** to generate a GitHub repos viewer:

```
Create a minimalistic GitHub Repo Viewer in HTML + JS:
  - Input field for a GitHub username
  - Button [Load Repos]
  - On click -> fetch
https://api.github.com/users/{username}/repos
  - Display the list of repos with name + link
  - Use plain JS (fetch API), no frameworks, single HTML file
```

📸 Try It Live

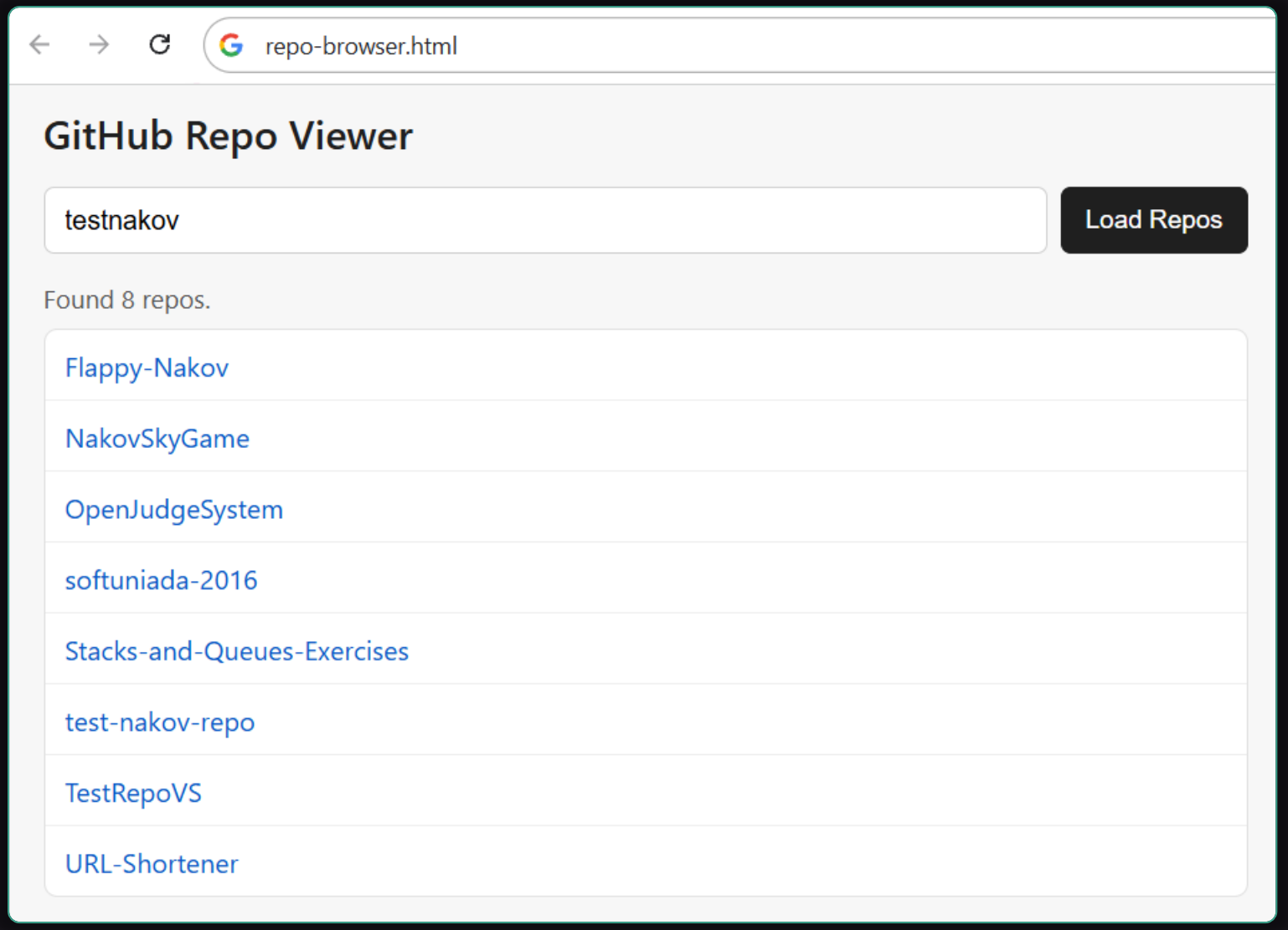
Paste prompt into AI → run in browser → test with any GitHub username

🔗 Endpoint Used

GET
https://api.github.com/users/{username}/repos

GitHub Repo Viewer App

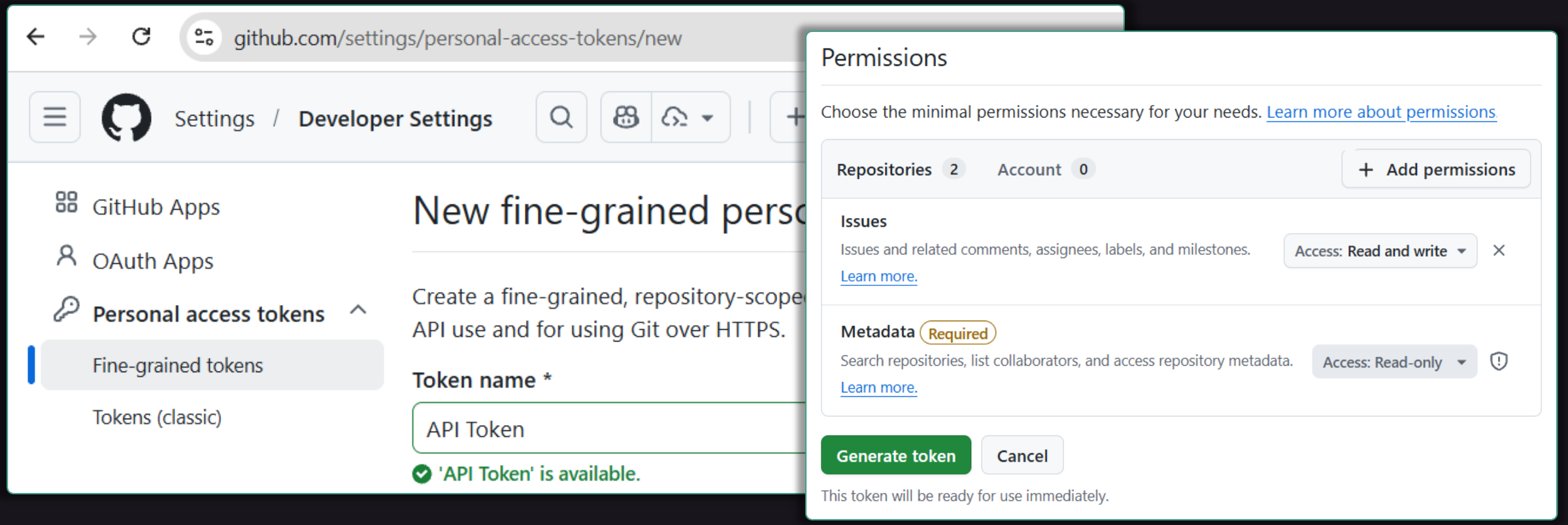
SoftUni AI



Creating an API Key in GitHub

SoftUni AI

- **API keys** in GitHub:
  - https://github.com/settings/personal-access-tokens



RESTful APIs with Authentication

SoftUni AI

🔑 Why Authentication and Authorization?

- Many APIs require an **auth token** to verify who you are
- **Personal Access Token (PAT)** in the `**Authorization**` header

🔑 Authorization Flow

- **Login** → returns JWT token
- **Next request** → send the token in the `**Authorization**` header

📡 Create a GitHub Issue (POST)

```
POST
https://api.github.com/repos/testnakov/test-nakov-repo/issues
Authorization: Bearer github_…
Content-Type: application/json
Body: { "title": "Bug: login fails", "body": "Steps …" }
```

```
201 Created + new issue JSON
```

🔑 **Keep tokens secret**. Never commit to Git!

Example — List / Create Issues App

SoftUni AI

- **AI prompt** to generate a GitHub Issues Browser app:

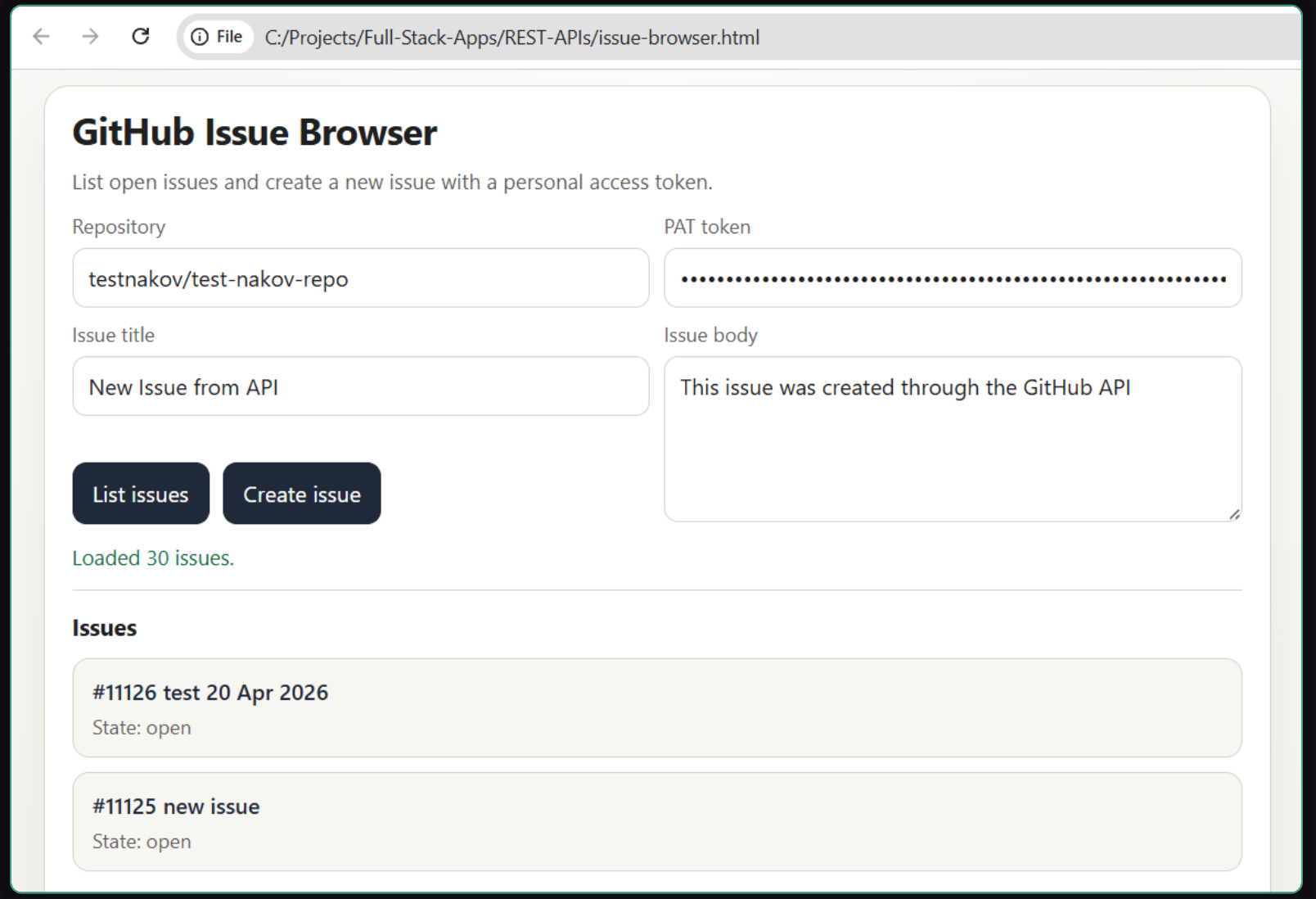
```
Create a minimalistic GitHub Issues browser in HTML + JS:
  - Input: GitHub repo (owner/name), auth token (PAT)
  - Button [List Issues] -> GET /repos/{owner}/{repo}/issues
  - Form: title + body + [Create Issue] button
  - POST /repos/{owner}/{repo}/issues with Bearer token
  - Keep it simple: single HTML file (HTML + CSS + JS)
```

✅ What This App Demonstrates?

- Authenticated GET request — listing existing issues
- Authenticated POST request — **creating a new issue** with JSON body
- Real-world GitHub API workflow with **token auth**

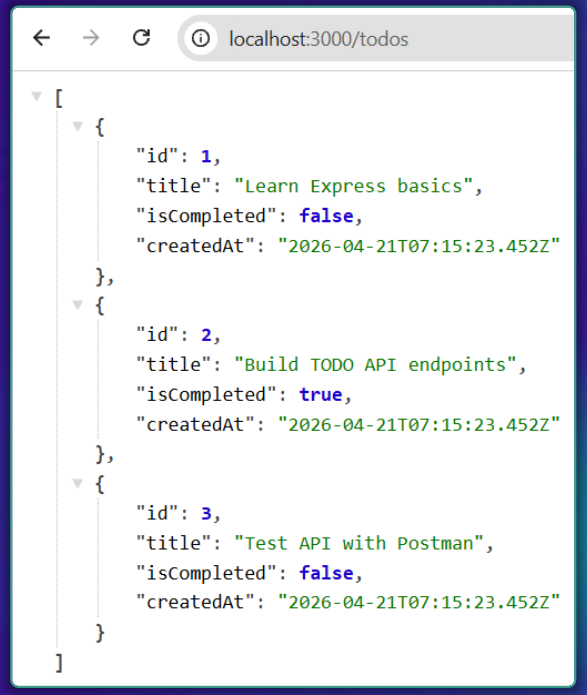
GitHub Issue Browser App

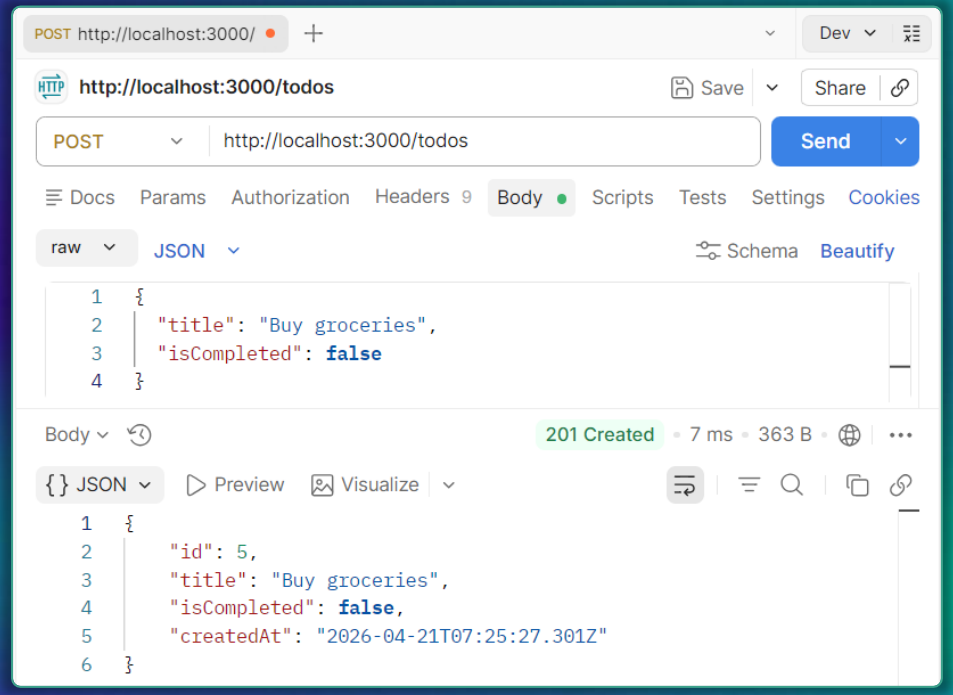
SoftUni AI

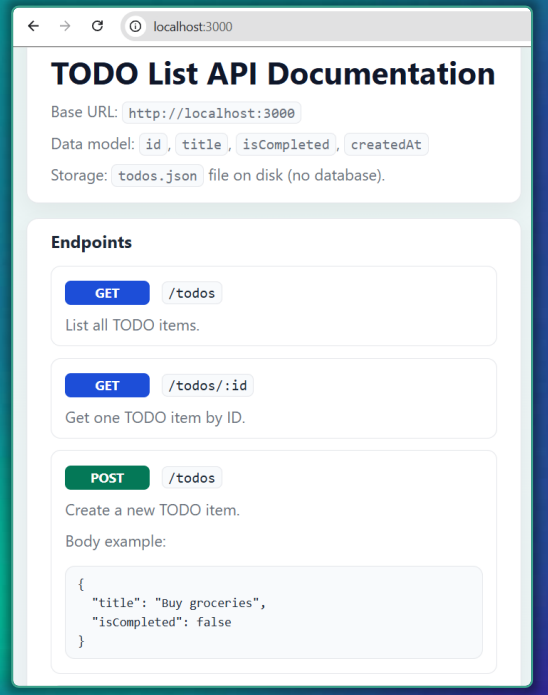


TODO List API in Node.js

Building a RESTful API with Node.js + Express, Testing with Postman



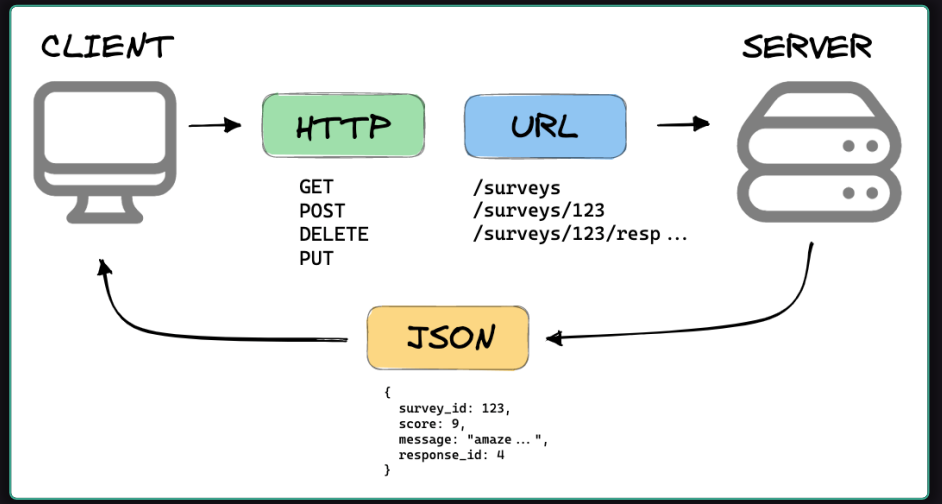




Creating a RESTful API Server

SoftUni AI

- Creating a RESTful API server in JavaScript
  1. Implement **API route handlers**:
     - Handle **GET /endpoint**
     - Handle **POST /endpoint**
  2. Listen to certain **port**, e.g. 8080



- Implementations:
  - **Node.js** + **Express**: simple, straightforward, flexible
  - **Next.js** API route handlers: established app framework

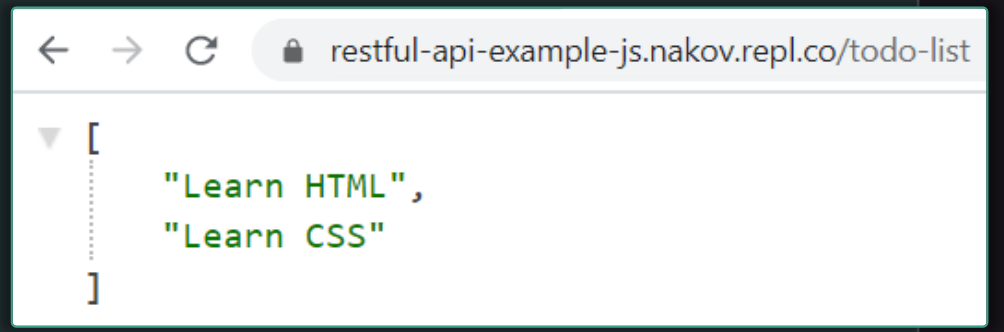
RESTful API Server in Node.js – Example

SoftUni AI

```
const express = require('express');
const app = express();
app.use(require('cors')());
app.use(require('body-parser').json());

let todoList = ['Learn HTML', 'Learn CSS'];

app.get('/todo-list', (req, res) => {
  res.json(todoList);
});

app.post('/todo-list', (req, res) => {
  todoList.push(req.body.todoText);
  res.sendStatus(200);
});

app.listen(8080);
```

```
npm install express cors
```

```
node ./api-server.js
```



🌐 **Minimalistic Demo**

- Note: this is a minimalistic demo: not for real use

TODO List API — Scope

SoftUni AI

- Implement a **TODO List API** in Node.js + Express

🗄️ Data Model

- **id** – unique number
- **title** – string
- **completed** – boolean
- **createdAt** – ISO date

🌐 API Docs @ Home Page

- Displays an HTML docs
- Lists all endpoints

📋 API Endpoints

- **GET /todos** – list all TODOs
- **GET /todos/:id** – get one TODO
- **POST /todos** – create
- **PATCH /todos/:id** – update
- **DELETE /todos/:id** – delete

💾 Storage Model

- Plain JSON file: `**todos.json**`
- No database – focus on HTTP & REST first

TODO List API — AI Prompt

SoftUni AI

- **AI prompt** to generate a TODO List API:

```
Implement a simple TODO List API in Node.js + Express:
  - Data model: id, title, isCompleted, createdAt
  - Storage model: JSON file: `todos.json` (no database)
  - Initially, load sample data (on first run)
  - API endpoints:
    - GET /todos – list all TODOs
    - GET /todos/:id – get one TODO
    - POST /todos – create
    - PATCH /todos/:id – update
    - DELETE /todos/:id – delete
  - At the home page implement HTML docs: list all endpoints
```

TODO List API — The App

SoftUni AI

localhost:3000/todos


```
[
  {
    "id": 1,
    "title": "Learn Express basics",
    "isCompleted": false,
    "createdAt": "2026-04-21T07:15:23.452Z"
  },
  {
    "id": 2,
    "title": "Build TODO API endpoints",
    "isCompleted": true,
    "createdAt": "2026-04-21T07:15:23.452Z"
  },
  {
    "id": 3,
    "title": "Test API with Postman",
    "isCompleted": false,
    "createdAt": "2026-04-21T07:15:23.452Z"
  }
]
```


localhost:3000

TODO List API Documentation

Base URL: `http://localhost:3000`

Data model: `id`, `title`, `isCompleted`, `createdAt`

Storage: `todos.json` file on disk (no database).

Endpoints

GET `/todos`

List all TODO items.

GET `/todos/:id`

Get one TODO item by ID.

POST `/todos`

Create a new TODO item.

Body example:

```
{
  "title": "Buy groceries",
  "isCompleted": false
}
```

POST http://localhost:3000/

http://localhost:3000/todos

Save | Share

POST | http://localhost:3000/todos | Send

Docs | Params | Authorization | Headers 9 | Body | Scripts | Tests | Settings | Cookies

raw | JSON | Schema | Beautify

```
{
  "title": "Buy groceries",
  "isCompleted": false
}
```

Body | 201 Created | 7 ms | 363 B

{} JSON | Preview | Visualize

```
{
  "id": 5,
  "title": "Buy groceries",
  "isCompleted": false,
  "createdAt": "2026-04-21T07:25:27.301Z"
}
```

Running the API Server

SoftUni AI

🗂️ Project Files Generated

- **server.js** — main entry point
- **todos.json** — JSON storage file (app data)
- **package.json** — npm scripts + dependencies
- **docs.html** — generated endpoint docs

▶️ Start the Server

- **npm install**
- **npm start**
- Output: *Server running at http://localhost:3000*

🔍 Quick Smoke-Tests in the Browser

- Open browser: http://localhost:3000/ → HTML docs page listing all endpoints
- GET http://localhost:3000/todos → JSON array of sample TODOs (200 OK)
- GET http://localhost:3000/todos/1 → single TODO object (200 OK)
- GET http://localhost:3000/todos/999 → { "error": "Not found" } (404 Not Found)

Testing with Postman

SoftUni AI

🧪 Test the TODO API with Postman

- Create **Postman collection**: "TODO API"
- **GET** http://localhost:3000/todos → 200 OK, JSON array
- **GET** http://localhost:3000/todos/1 → 200 OK + TODO object
- **GET** http://localhost:3000/todos/999 → 404 Not Found + error JSON
- **POST** http://localhost:3000/todos + JSON body → 201 Created + TODO object
- **POST** http://localhost:3000/todos + empty body → 400 Bad Request
- **PATCH** http://localhost:3000/todos/1 + partial JSON → 200 OK, updated TODO
- **PATCH** http://localhost:3000/todos/999 + partial JSON → 404 Not Found
- **DELETE** http://localhost:3000/todos/1 → 200 OK or 204 No Content
- **DELETE** http://localhost:3000/todos/999 + partial JSON → 404 Not Found

RESTful API Design Guidelines

Resource Naming, HTTP Methods, Status Codes, JSON, Filtering, Pagination

Resource Naming

SoftUni AI

📌 Use Nouns, Not Verbs

- ✅ **GET /users, POST /users**
- ❌ **GET /getUsers**
- Use plural nouns:
  /products, **/orders**, **/todos**
- Use lowercase + hyphens:
  /blog-posts (not **/blogPosts**)

🔑 Key Naming Rules

- Collections → plural nouns:
  /todos, **/users**, **/repos**
- Single resource → noun + ID:
  /todos/7, **/users/42**
- Sub-resources:
  /posts/5/author, **/posts/5/visibility**

🔗 Nested URLs for Relations

- **/users/5/orders** → orders of user #5
- **/users/5/todos** → TODOs of user #5
- Keep nesting shallow — max 2 levels deep

HTTP Methods Mapping

SoftUni AI

🔁 CRUD → HTTP Mapping

- Create → **POST /users**
- Read (list) → **GET /users**
- Read (one) → **GET /users/5**
- Read (report) → **GET /users/stats**
- Update (partial) → **PATCH /users/5**
- Update (full) → **PUT /users/5**
- Update (state) → **PUT /posts/5/visibility → { "hidden": true }**
- Delete → **DELETE /users/5**

✅ GET Is Always Safe

- Never modifies data — safe to retry
- Idempotent: same result every call

⚠️ Modifying methods

- **POST** — creates; not idempotent
- **PATCH** — partial update
- **PUT** — replaces the entire object
- **DELETE** — removes; idempotent

HTTP Status Codes Guidelines

SoftUni AI

✅ 2xx Success

- **200 OK** — success
- **201 Created** — after POST
- **204 No Content** — after DELETE
- **202 Accepted** — async job started
- **206 Partial Content** — bytes range

⚠️ 4xx Client Errors

- **400 Bad Request** — invalid input
- **401 Unauthorized** — no auth / bad auth
- **403 Forbidden** — action not allowed
- **404 Not Found** — missing resource
- **409 Conflict** — duplicate (on create / edit)

🔥 5xx Server Errors

- **500 Internal Server Error** — unhandled failure / exception
- **503 Service Unavailable** — server overloaded
- **502 Bad Gateway** / **504 Gateway Timeout** – proxy failed to access upstream
- **501 Not Implemented** — functionality unavailable

JSON Responses

SoftUni AI

📦 Always Return JSON for Data Endpoints

- Set **Content-Type: application/json** on all responses
- Keep **response shape** consistent — same fields, same types
- Use **camelCase** for field names: **createdAt**, **userId**, **isActive**

📄 Single Resource

```
{ "id": 1, "title": "Sleep", "completed": false }
```

📋 Collection

```
[ { "id": 1, ... }, { "id": 2, ... } ]
```

❌ Error Response

```
{ "error": "TODO not found", "code": 404 }
```

✅ Best Practices

- Wrap lists in object: { data: [...], total: 42 }
- Include error details in error responses

Pagination, Filtering and Versioning

SoftUni AI

📄 Pagination

- **GET /api/v1/users?page=2&pageSize=20**
- Prevents returning 10 000 items at once
- Return **data** + **count**: { data: [...], total: 847 }

🔀 Versioning

- Keep old clients working when API changes
- **/api/v1/users** → **/api/v2/users**

🔍 Filtering & Sorting

- Filter: **GET /users?role=admin&active=true**
- Sort: **GET /users?sort=createdAt&order=desc**
- Combined: **?role=admin&sort=name&page=1**

💡 Full Example

- GET /api/v1/users
  - ?role=admin
  - &active=true
  - &sort=createdAt&order=desc
  - &page=2&pageSize=20

Next.js Basics and APIs

Next.js Apps Structure, API Route Handlers

What is Next.js?

SoftUni AI

⚡ Next.js — React-Based Full-Stack Framework

- Built on top of **React** by **Vercel**
- Used by Netflix, TikTok, Nike, and many more
- https://nextjs.org

📁 Key Features

- File-based **routing** (pages in **app/** folder)
- **Server** and **client** components
- Built-in **API routes** — back-end in same project
- **Server-side rendering** (SSR)
- Image optimization, built-in fonts, middleware

🚀 Why Next.js for Back-End APIs?

- One project, one deploy — **front-end + back-end** together
- API routes live at **app/api/*/route.ts** — standard file-based routing
- Works seamlessly on **Vercel**, **Netlify**, **Railway** — serverless or server-based
- **TypeScript** first-class – great developer experience

Create a Next.js Project

SoftUni AI

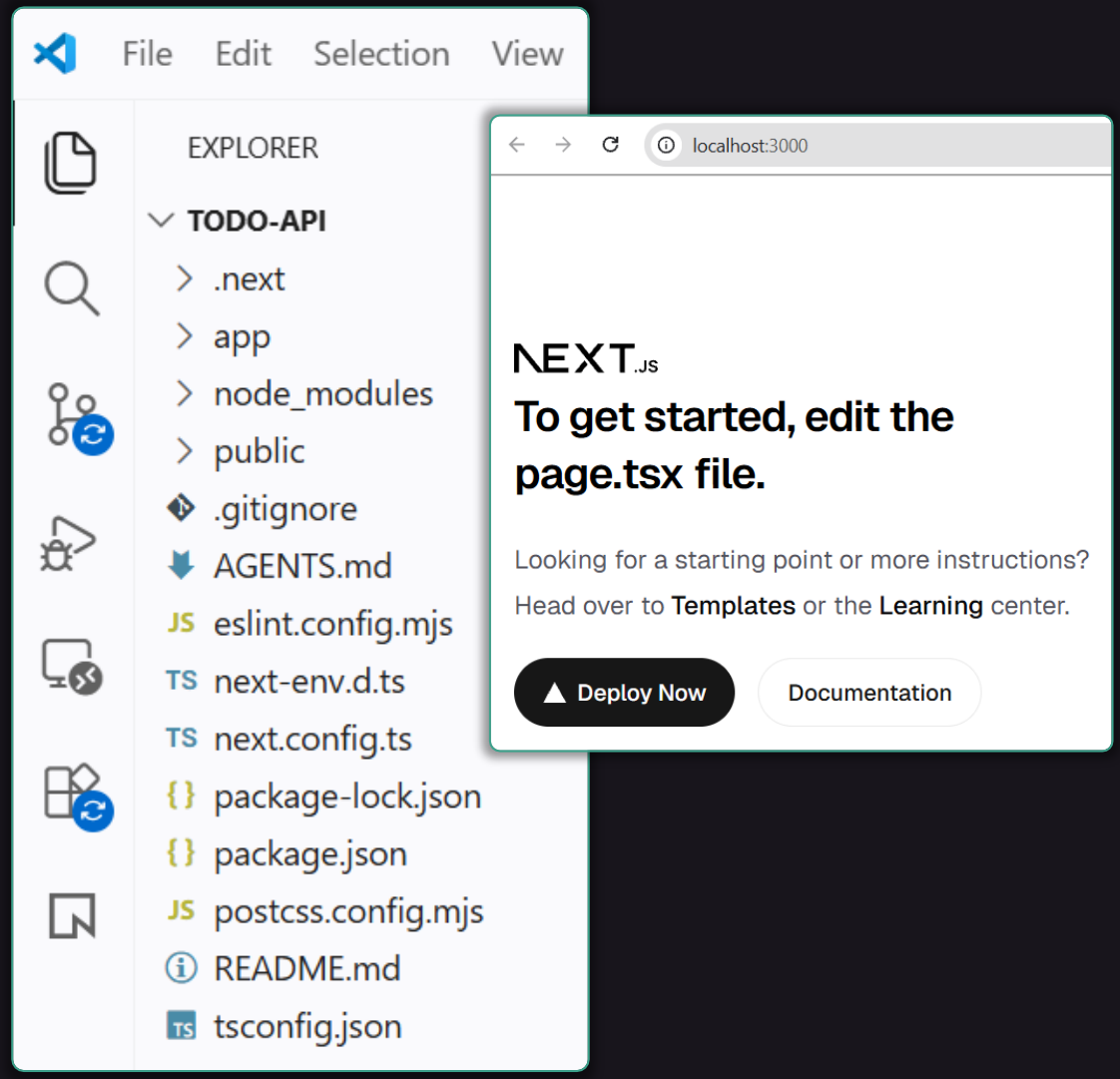
🚀 Initialize a Next.js Project

```
npx create-next-app@latest todo-api
```

- Answer the prompt:
  - *Yes, use recommended defaults*

▶️ Run the `dev` Server

- **npm run dev**
- Open: http://localhost:3000
- Hot reload — changes appear instantly



Next.js Project Structure

SoftUni AI

📁 Project Structure Overview

- **app/page.tsx** — home page — **/**
- **app/layout.tsx** — root layout wrapping all pages
- **app/api/** — all API route handlers live here
- **public/** — static assets (images, fonts, icons)

📁 API Route Handlers

- **app/api/hello/route.ts** — API route handlers — maps to **/api/hello**

API Route Handlers

SoftUni AI

📡 Route Handlers in Next.js

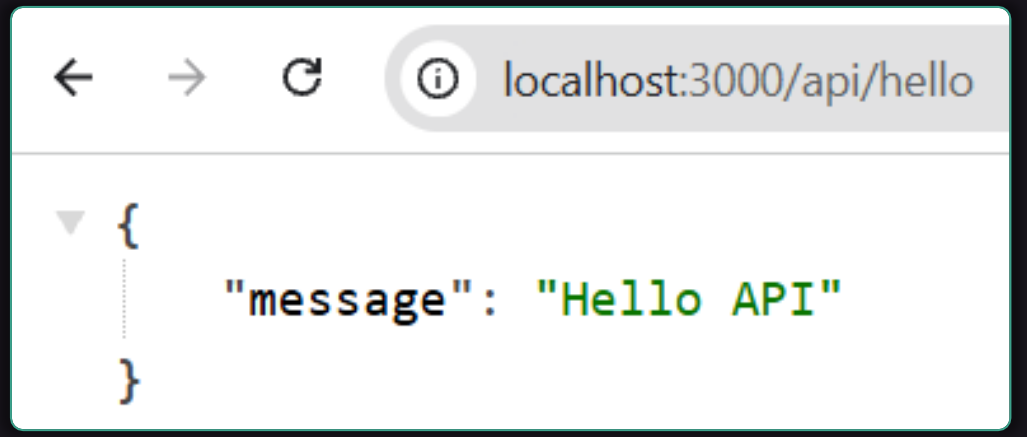
- Live in **app/api/.../route.ts**
- Export HTTP methods as named functions:
  - **export async function GET() { ... }**
  - **export async function POST(req) { ... }**
- Return **Response.json({ data })**

🔧 Accessing Request Data

- URL params: **function GET(req, { params })**
- Query string: **new URL(req.url).searchParams**
- Request body: **const body = await req.json()**
- Headers: **req.headers.get('Authorization')**

app/api/hello/route.ts

```
export async function GET() {
  let result = {"message":
  "Hello API"};
  return Response.json(result);
  // 200 OK
}
```



Example — Simple API Endpoint

SoftUni AI

💡 AI Prompt

- Create API route: **GET /api/hello**
  - Returns **{ msg: 'Hello, <name>!' }**
  - Read **name** from query param **?name=Svetlin**
  - Default to '**API**' if not provided

📄 app/api/hello/route.ts

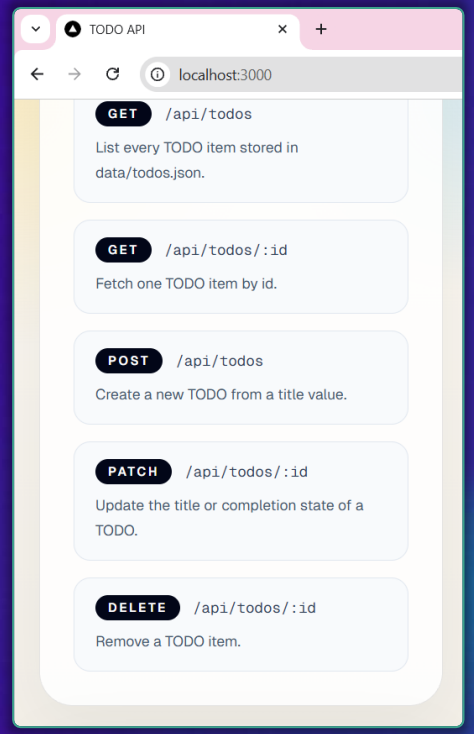
```
export async function GET(req: Request) {
  const url = new URL(req.url);
  const name = url.searchParams.
  get('name') ?? 'API';
  return Response.json({ msg: `Hello,
  ${name}!` });
}
```

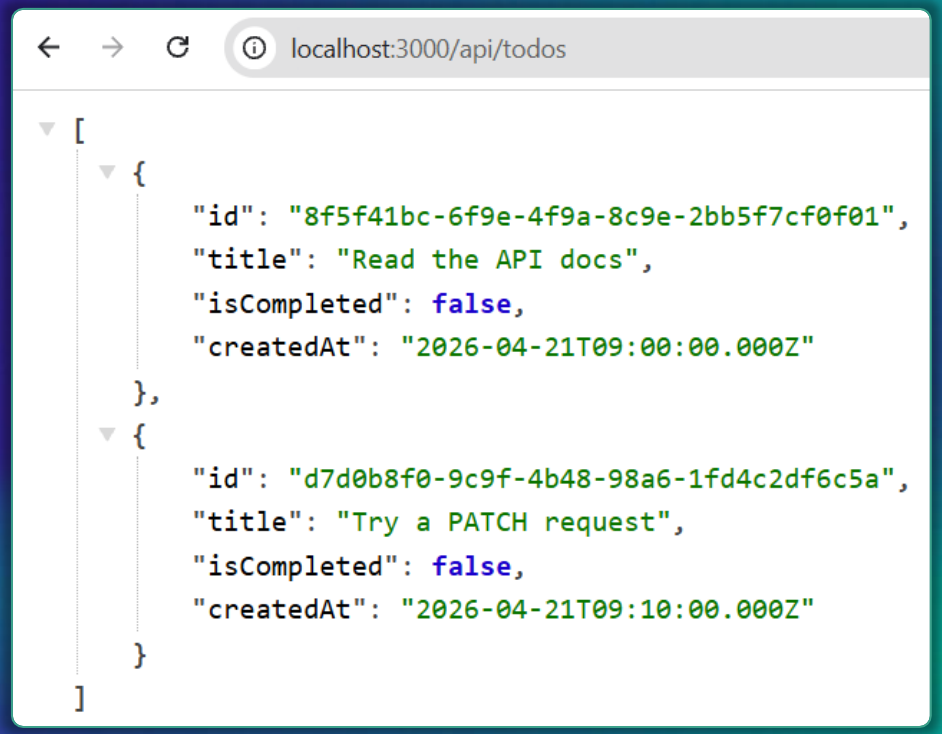
▶️ Test the API Endpoint: /api/hello

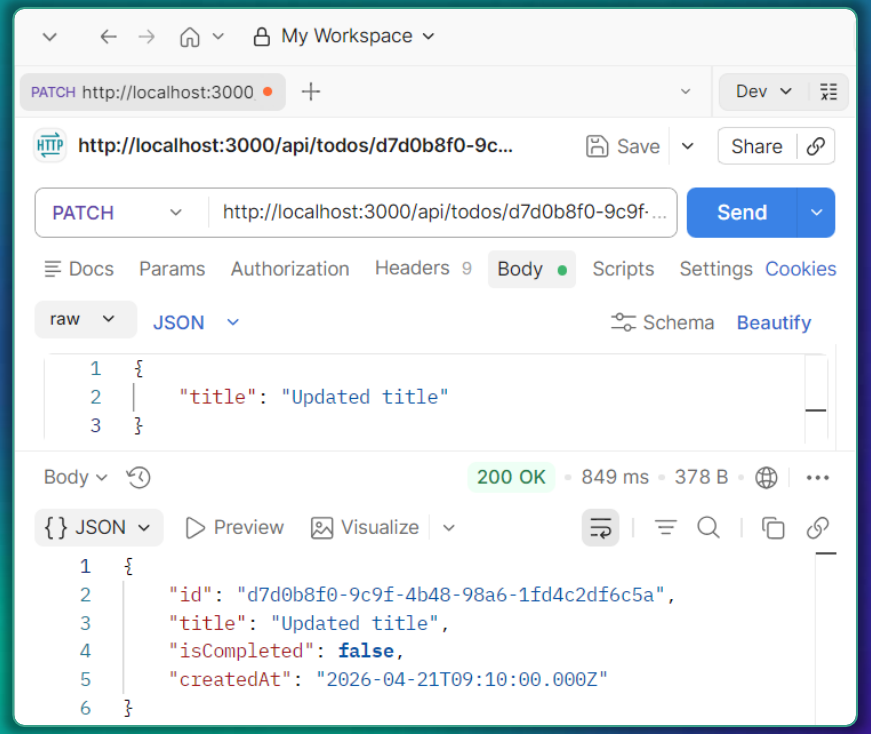
- http://localhost:3000/api/hello → { message: 'Hello, API!' }
- http://localhost:3000/api/hello?name=SoftUni → { message: 'Hello, SoftUni!' }

TODO List API with Next.js

Implementing CRUD Endpoints with Next.js







TODO API with Next.js: The Plan

SoftUni AI

📋 Rebuild the TODO API with Next.js

- Same **CRUD endpoints** — now as **Next.js API route handlers**
- **GET / POST /api/todos** and **GET / PATCH / DELETE /api/todos/[id]**

🗂️ Project Structure

- **app/api/todos/route.ts** (GET, POST)
- **app/api/todos/[id]/route.ts** (GET, PATCH, DELETE)
- **lib/todos.ts** — data access helper
- **data/todos.json** — storage file

💾 Storage

- **data/todos.json** (no DB yet)
- **Todo { id, title, isCompleted, createdAt }**
- Initially load **sample data**

🏠 **App Home Page**: show simple API docs (describe the endpoints)

TODO List API — AI Prompt

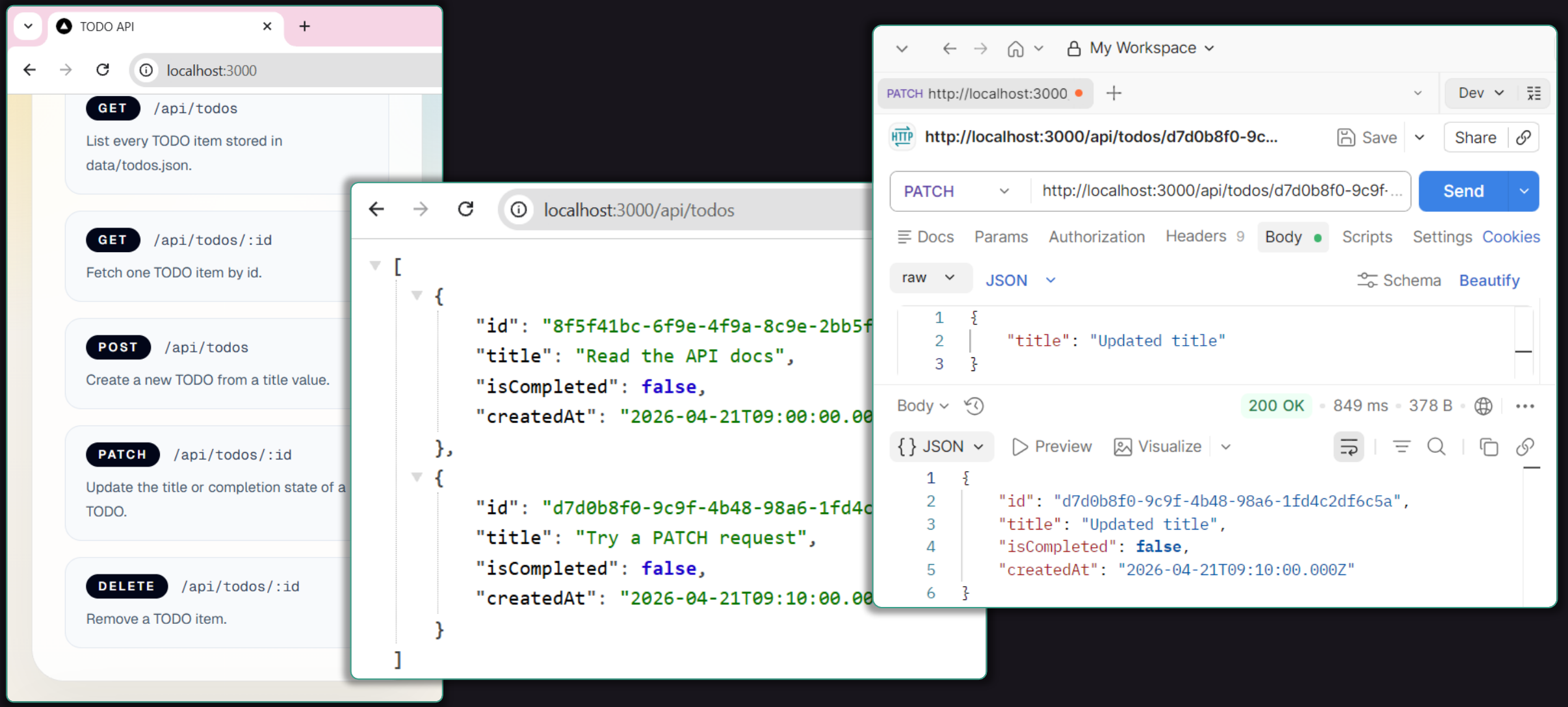
SoftUni AI

- **AI prompt** to generate a TODO List API in Next.js:

```
Implement a simple TODO List API with Next.js:
  - Data model: id (number), title, isCompleted, createdAt
  - Storage model: JSON file: `data/todos.json` (no database)
  - Initially, load sample data (on first run)
  - API endpoints:
    - GET /api/todos – list all TODOs
    - GET /api/todos/:id – get one TODO
    - POST /api/todos – create
    - PATCH /api/todos/:id – update
    - DELETE /api/todos/:id – delete
  - At the home page show simple API docs: describe endpoints
```

TODO List API in Next.js

SoftUni AI



Project Structure

SoftUni AI

📁 Folder Structure

app/ ← Next.js pages and API routes
  page.tsx ← home page (API docs)
  api/todos/ ← API route handlers
    route.ts ← GET, POST /api/todos
    [id]/route.ts ← GET, PATCH, DELETE

📦 Supporting Files

lib/
  todos.ts ← data access layer
data/
  todos.json ← JSON storage
package.json ← NPM scripts

🔑 Key Design Principles

- All data access goes through **lib/todos.ts** — no direct file I/O in route handlers
- **Route handlers** stay thin: validate input, call lib, return Response.json(...)
- Keep **todos.json** in the **data/** folder — easy to replace with a real DB later

Testing with Postman

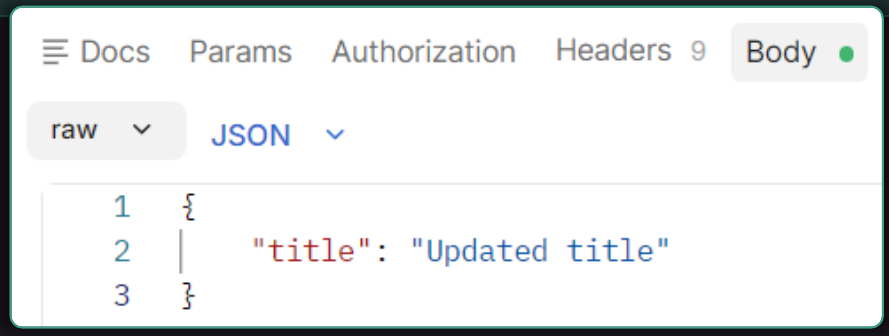
SoftUni AI

🧪 Postman Tests for "TODO Next.js API"

- Create Postman collection: "TODO Next.js API"
- **GET** /api/todos → 200 OK, JSON array
- **POST** /api/todos + JSON body → 201 Created, new TODO object
- **GET** /api/todos/1 → 200 OK, single TODO
- **PATCH** /api/todos/1 + { "completed": true } → 200 OK, updated TODO
- **DELETE** /api/todos/2 → 200 OK or 204 No Content
- **GET** /api/todos/999 → 404 Not Found, error JSON

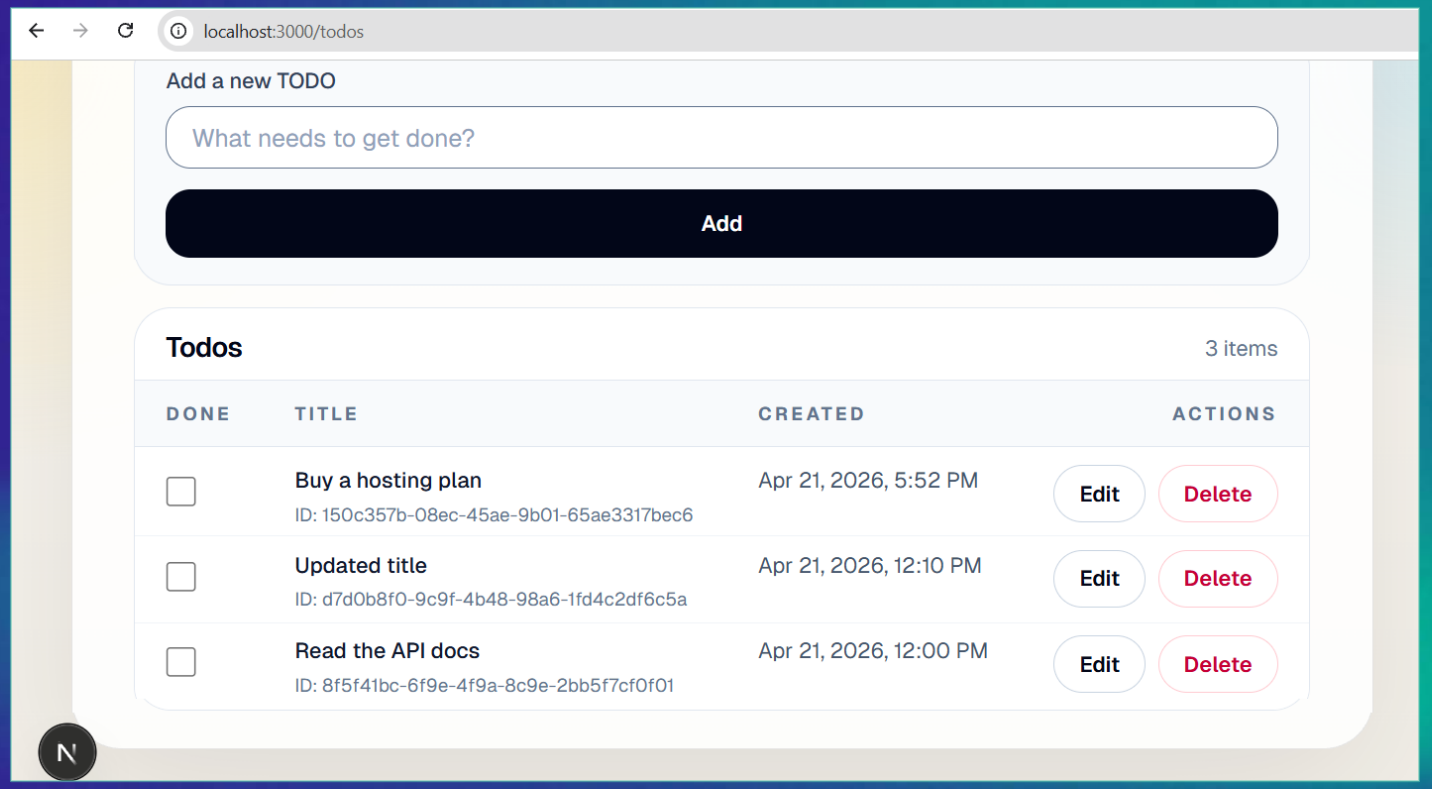
💡 Tips for Postman Testing

- Body → raw → JSON



TODO List Client App

Consume the API from a React UI: Implementing View, Add, Edit, Delete



Next.js Client Apps

- **Next.js** is powerful platform to build **full-stack apps**
- **Server-side** logic (back-end)
  - Runs at the Next.js server, e.g. in Netlify
  - API route handlers, server-rendered React components
- **Client-side** logic (front-end)
  - Traditional client-side React components
  - Client components interact with the server-side logic
- **Mixing** server-side and client-side components

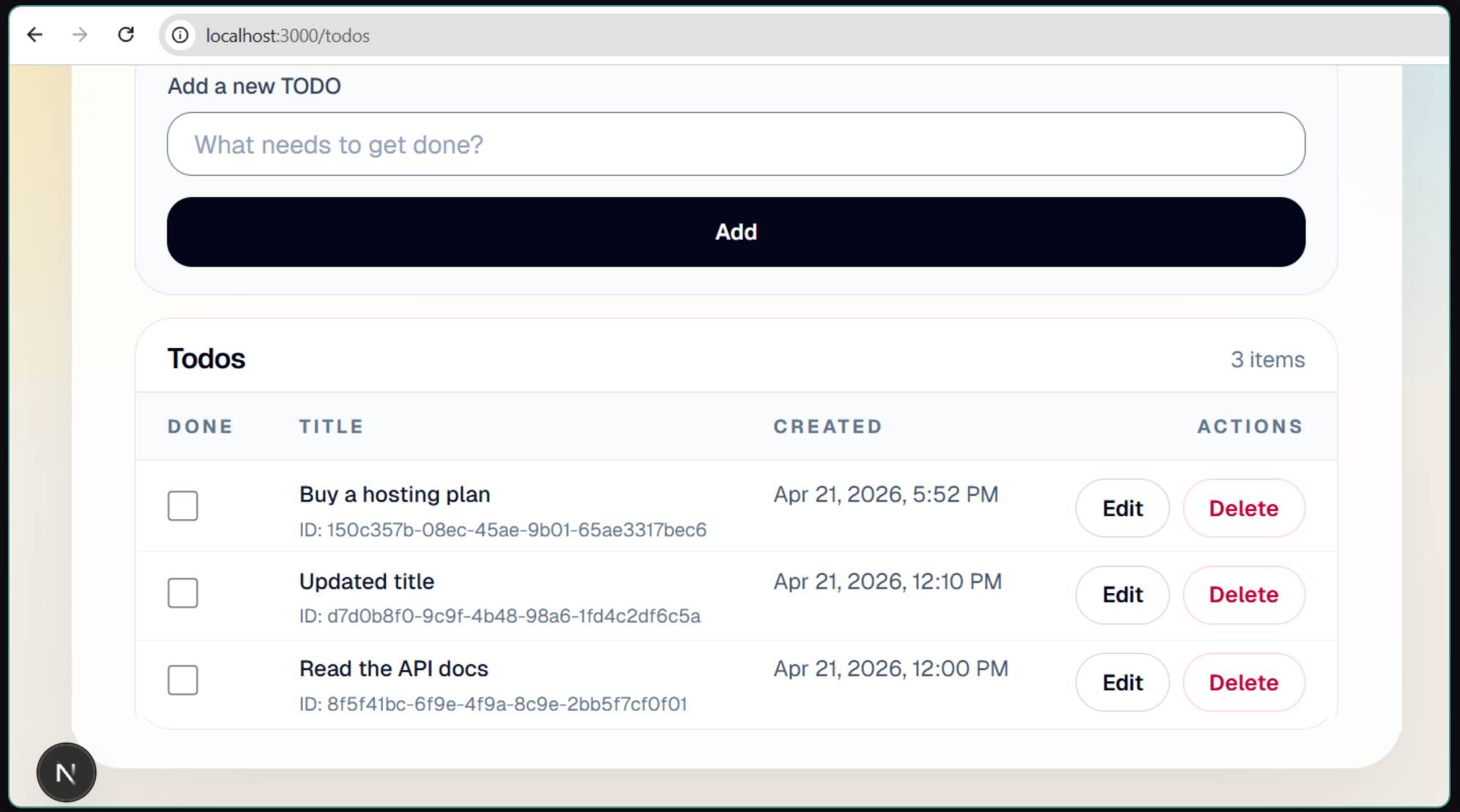
TODO List Client App — AI Prompt

- **AI prompt** to generate a **TODO List client page** in **Next.js**:

```
Implement a TODO List client page in Next.js:
  - A new page: /todos — in the same Next.js project
  - Use the existing API: fetched from `/api/todos`
  - View the list of TODOs — in a table
  - Toggle `Completed` — checkbox for each row
  - Edit title inline — [Edit] button for each row
  - Delete a TODO — [Delete] button for each row
  - Add a new TODO — input field + [Add] button
```

TODO List Client App – Result

SoftUni AI



The Client App – Implementation

SoftUni AI

🔄 Architecture: Same Project, Two Layers (API + Client)

- Browser → **/todos** page (React client component) → **fetch()** → **/api/todos** (Next.js API route) → **data/todos.json**
- Full-stack in a single project: all code lives in one repo, one `**npm run dev**`
- Later: **add** auth, replace JSON with a **database**, deploy to **Netlify**

⚙️ Technical Implementation

- Client **React component** — must add '**use client**' at the top
- Uses **useState()** to keep the TODOs list locally
- Uses **useEffect()** to fetch todos on page load
- Uses **fetch()** to **POST**, **PATCH**, **DELETE** via the API

Authentication & Authorization

JWT Tokens, Password Hashing

Authentication vs. Authorization

SoftUni AI

🔑 Authentication: Who Are You?

- Verify the **identity** of the person or system
- Authentication mechanisms: **email** + **password**, **OAuth**, **magic link**
- Result: a **session cookie** or a signed **auth token** (JWT)
- In our app: **POST /api/auth/login** → returns **JWT** token
- Rule: **auth must happen first** — you can't authorize without identity

🛡️ Authorization: What Can You Do?

- Check **permissions** for the authenticated user
  - Based on **roles**, **scopes**, or resource **ownership**
  - Examples: Is this user an **admin**? Can they delete others' TODOs?
- In our app: only the **owner** can see/edit their own TODOs
- Rule: **authenticate** first, then **authorize** for each action

JWT Tokens

- **JWT** (JSON Web Token) is modern standard for **authentication** and **authorization**
  - A JWT token holds info about the **currently logged-in user**: **user_id** / **username**, **roles** and **permissions** (optional)
  - Holds a **digital signature**, created by the token issuer
- The **JWT token** is:
  - **Issued at login** (after credentials are verified)
  - **Sent with each request** to identify and authorize the user
  - **Removed** from the client at **logout** (to forget logged-in user)

What is a JWT Token?

SoftUni AI

🔐 JWT Structure

- JWT = **JSON Web Token** (RFC 7519)
- A self-contained, cryptographical token
- Three parts separated by dots (**.**):
  - **Header.Payload.Signature**
- Example: **eyJhb... . eyJzW... . n2Hw...**

📦 JWT Payload (Decoded)

- The payload contains 'claims' about the user:

  { **"sub": "123", "email": "peter@email.com",**
  "iat": 1713350400, ← issued at
  "exp": 1713436800 ← expires at
  }

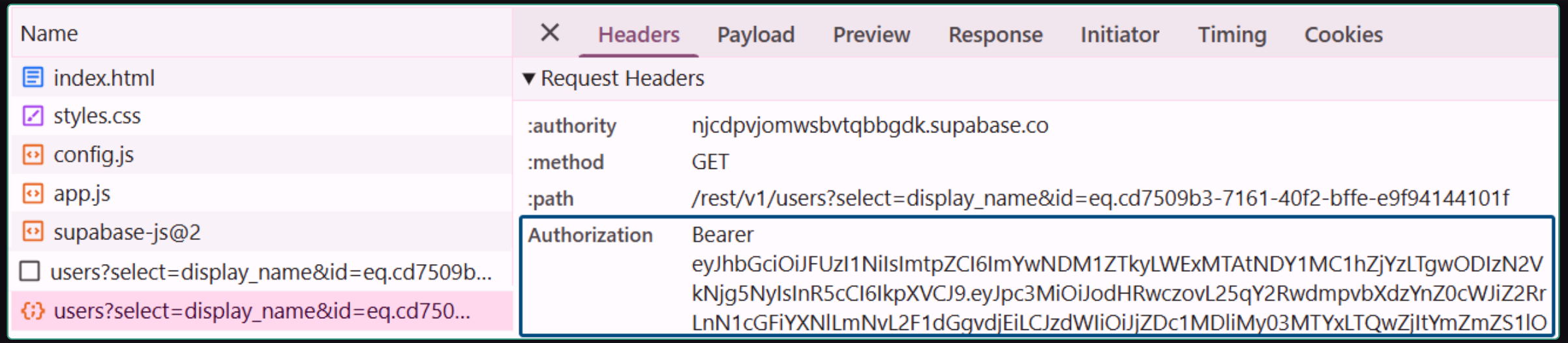
- Anyone can **decode JWT** — never store secrets in it!

✅ Why JWT?

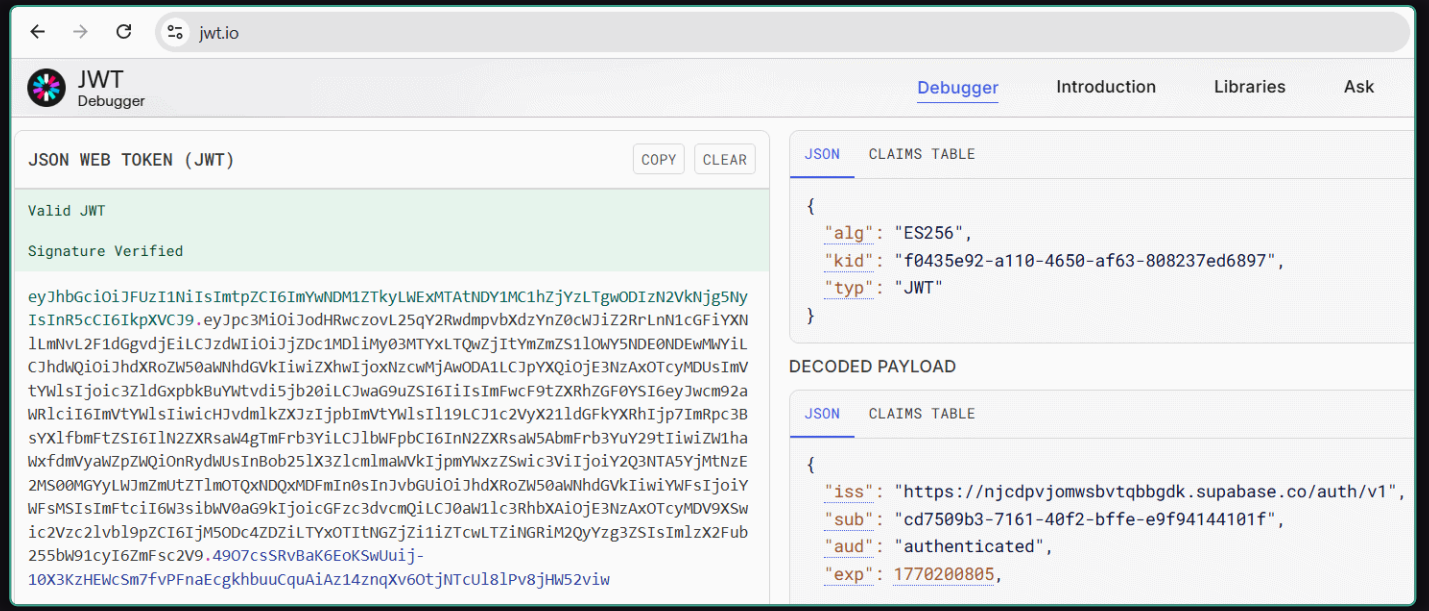
- **Stateless** — the server doesn't need to store session state (works great for serverless)
- **Self-contained** — the token carries all necessary information about the user
- **Portable**, **standard** — works across Web apps, mobile apps, and third-party clients

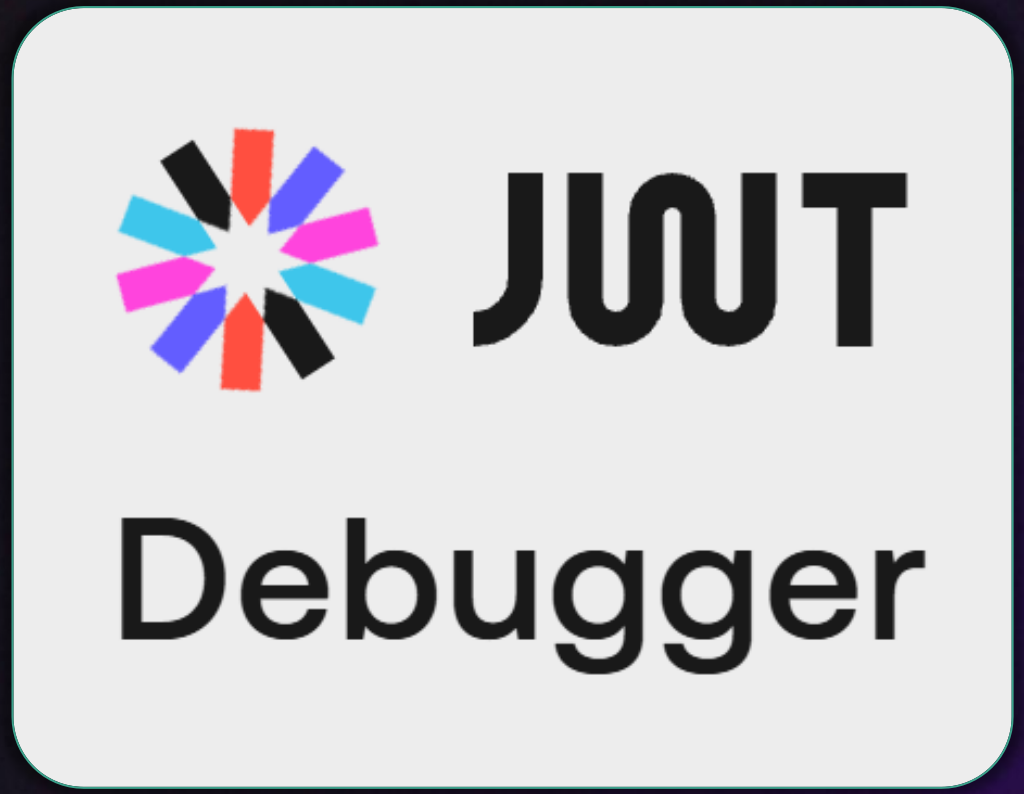
Inside JWT Tokens

SoftUni AI



header	payload	signature
{ "alg": "ES256", "typ": "JWT" }	{ "sub": "jonny86", "iat": 1516239022 }	KMUFsIDTnFmyG3nMiGM6H9FNFUROf3wh7S...





JWT Tokens

Live Demo

https://www.jwt.io

RESTful APIs with Auth

SoftUni AI

🔑 Typical Authentication Flow (Auth)

1. Client sends **auth claims** (username + password): **POST /api/auth/login**
2. Claims are **checked** by the server: OK → issue **JWT** token. Not → **error**
3. The **JWT token** is saved by the app for the next requests

🔄 Typical Request Flow with Auth (Authorized Requests)

1. Client sends request with **Authorization: Bearer <token>** header
2. Server extracts the **token** and **verifies** the signature
3. Server reads the **user ID** from the token payload
4. Server **checks permissions**: is this user allowed to do this action?
5. If **OK** → process the request. If not → return **401** or **403**

Hashing Passwords

SoftUni AI

⚠️ Never Store Plain Text Passwords!

- Storing **passwords as plain text** is a **critical security flaw**
- Always hash passwords with a **one-way algorithm**:
  - **bcrypt** — most common (npm: **bcryptjs**)
  - **argon2** — modern, for new projects
- Store only the **hash** + **salt** — the password is never stored

💡 Salt Rounds Parameter

- **bcrypt.hash(password, 10)**
- 10 == cost factor (salt rounds)
- Higher rounds = stronger hash = slower computation
- 10 is a good default

🔧 BCrypt: Register User

```
const hash = await
  bcrypt.hash(password, 10);
// Save hash to users.json / database
```

🔧 BCrypt: Login User

```
const ok = await
  bcrypt.compare(password, hash);
if (!ok) return 401 (Unauthorized)
```

BCrypt Hashes

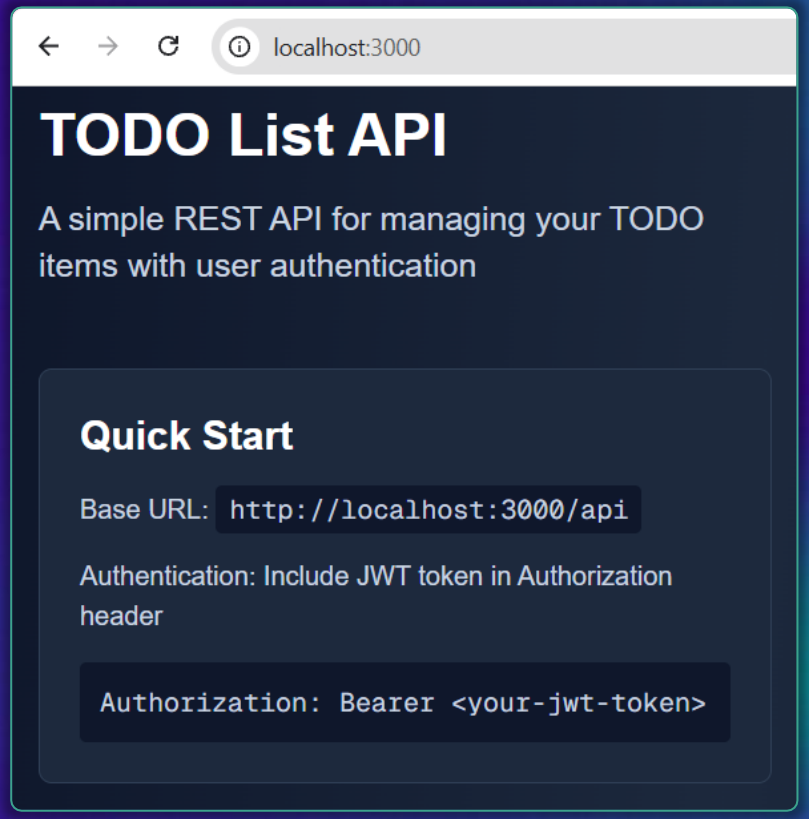
https://bcrypt-generator.com

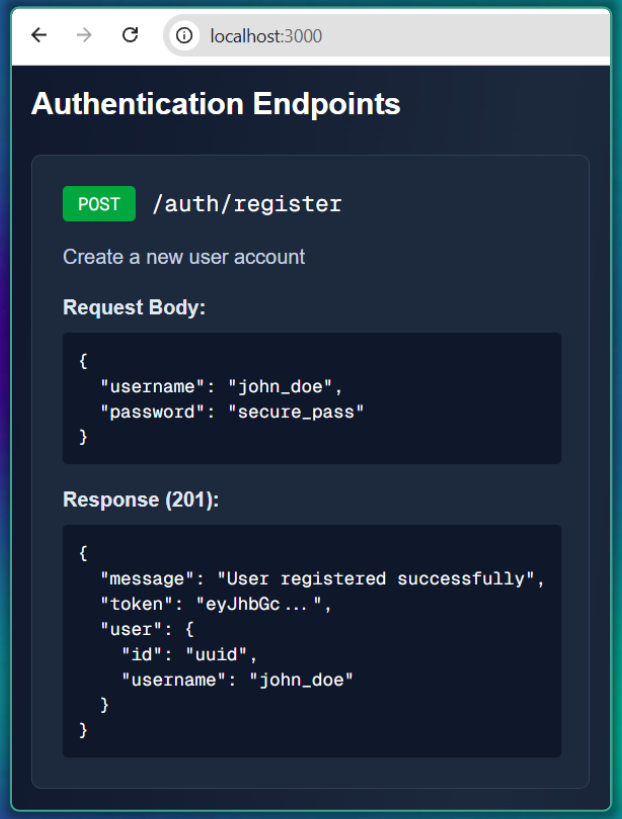
```
$2a$10$EG5S0thG2oaPBzXyfK/qS.J
Aqc1gawd4Ik5O8SdcjIjUgZLSkd7Wi
```

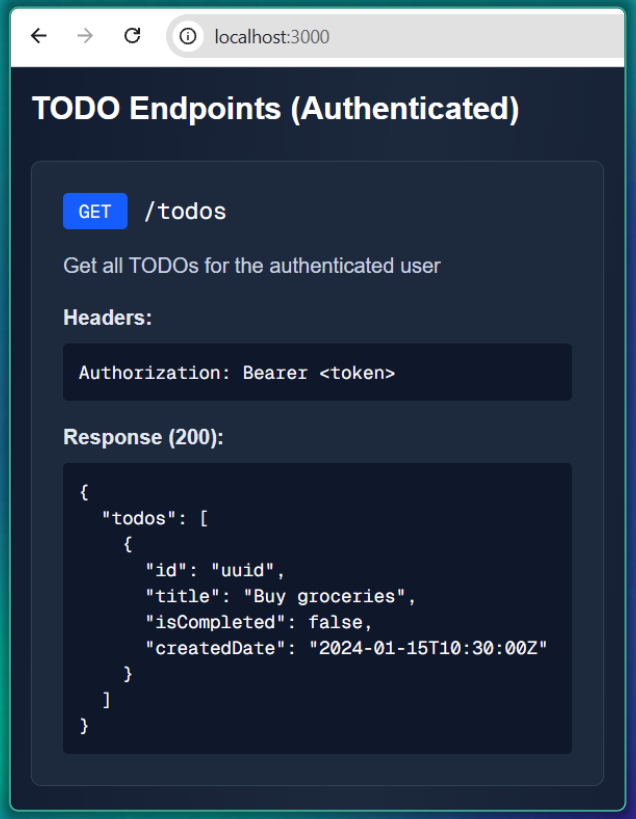
Live Demo

TODO List API with JWT Auth

Register, Login, Logout, Protected Endpoints







TODO List API with JWT Auth

SoftUni AI

- We want to build a **TODO List API server** with **auth**
  - Users **register** / **login** and keep their **own TODO items**
  - Auth endpoints: **register** / **login** → return **JWT token**
  - Data endpoints: **list** / **view** / **create** / **edit** / **delete** TODO
    - Authenticate with a JWT token
  - Data model:
    - **Users**: username, password hash, TodoItem[]
    - **TODO items**: title, isCompleted, createdDate

TODO List API with JWT Auth in Next.js

SoftUni AI

- AI prompt to build **TODO List API** with **JWT Auth** in **Next.js**

Implement a **TODO List API server** as simple **Next.js** app:
- Users **register** / **login** and keep their **own TODO items** on the server
- Auth endpoints: **register** / **login** → return **JWT token**
- Data endpoints: **list** / **view** / **create** / **edit** / **delete** TODO (for authenticated users, with JWT token)
- Data model: keep data in a simple file `**data/app-data.json**`
  - **Users**: username, password hash (bcrypt), TodoItem[]
  - **TODO items**: title, isCompleted, createdDate
- Generate simple **API docs** at the home page: describe the endpoints

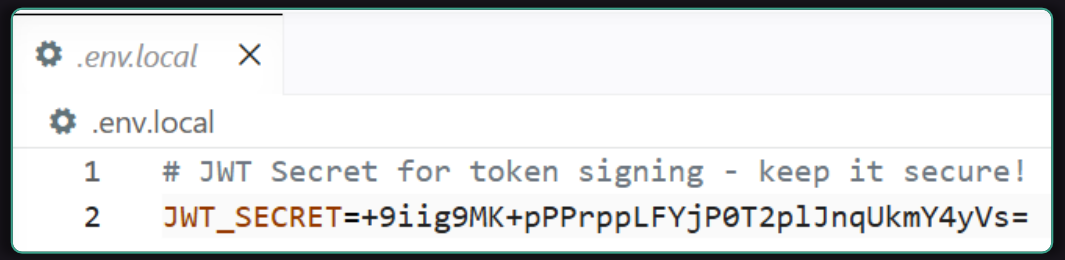
Security Warning: JWT_SECRET

SoftUni AI

- Most apps use an environment config value **JWT_SECRET**:

`.env.local` *(your private config file)*

```
JWT_SECRET=random_long_secret
```



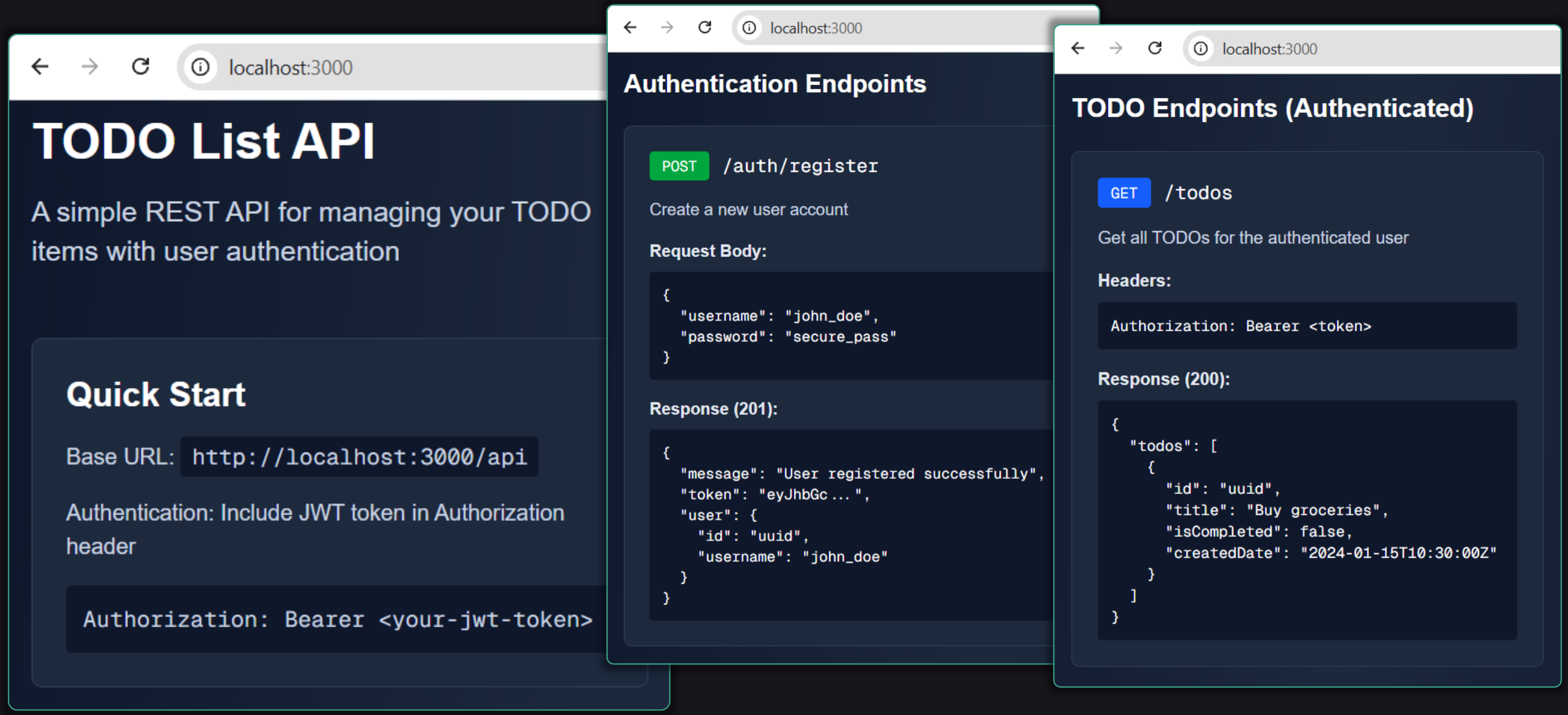
- Critical **security flaw**: if **JWT_SECRET** leaks, attackers can **login without a password**!
- AI prompt to **configure the JWT_SECRET**:

Generate a **random `JWT_SECRET`** in **`.env.local`**

- Even better: configure your key **manually** (without AI agent)

TODO List API with JWT Auth

SoftUni AI



Inside the TODO List API with Auth

SoftUni AI

🆕 Auth Endpoints

- **POST /api/auth/register** — create new user, return JWT
- **POST /api/auth/login** — verify credentials, return JWT
- **Logout** — no endpoint at the server (client drops token)
- **GET /api/auth/me** — return current user from JWT
- Storage: **data/app-data.json** (email + passwordHash)

🔒 Protected Data Endpoints

- **GET**, **POST /api/todos**
- **GET**, **PATCH**, **DELETE /api/todos/:id**
- All **/api/todos** endpoints now require **JWT auth**
  - **Authorization: Bearer <jwt_token>**
  - Missing or invalid token → **401 Unauthorized**
- Each user sees only their own TODOs
  - JWT token holds the **user ID**

Testing Auth with Postman

SoftUni AI

📋 Step 1 — Register

- **POST http://localhost:3000/api/auth/register**
  - Body: **{ "username": "steve", "password": "pass123" }**
  - Expected: **201 Created** + **{ "token": "eyJ..." }**
- Copy the **token** for the next requests
- Try registering the same username again → **409 Conflict**

📋 Step 2 — Use the Token

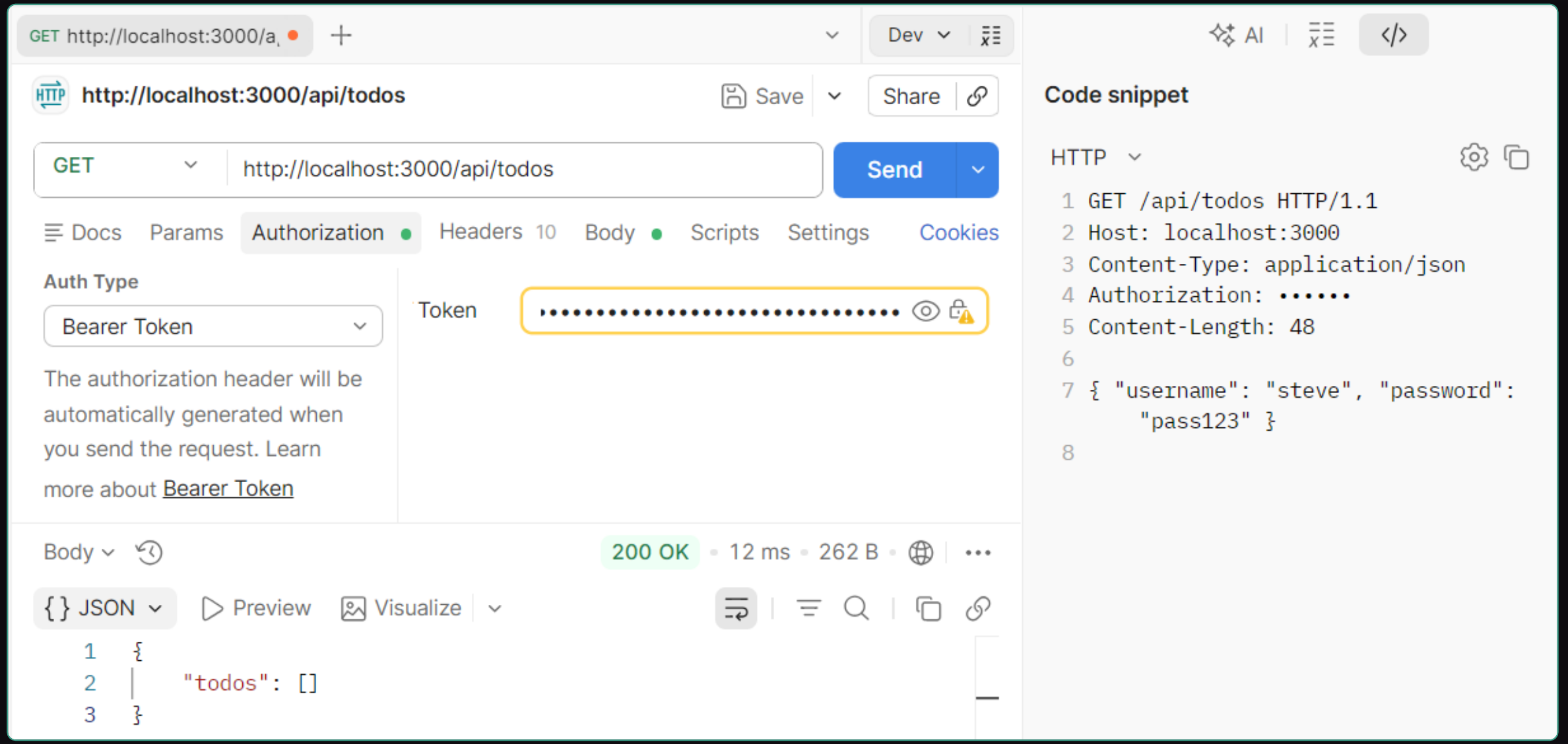
- **GET http://localhost:3000/api/todos**
  - **Authorization: Bearer eyJ...**
  - Expected: **200 OK** + **{ "todos": [] }**
- Without the Authorization header:
  - Expected: **401 Unauthorized** + { error: 'Unauthorized' }

📋 Step 3 — Full Auth Flow

- **POST /api/todos** with token → **201 Created** (TODO created for this user)
- **GET /api/todos** with same token → **200 OK**, shows your TODOs
- Login as a DIFFERENT user → **GET /api/todos** → sees empty list (own TODOs only)

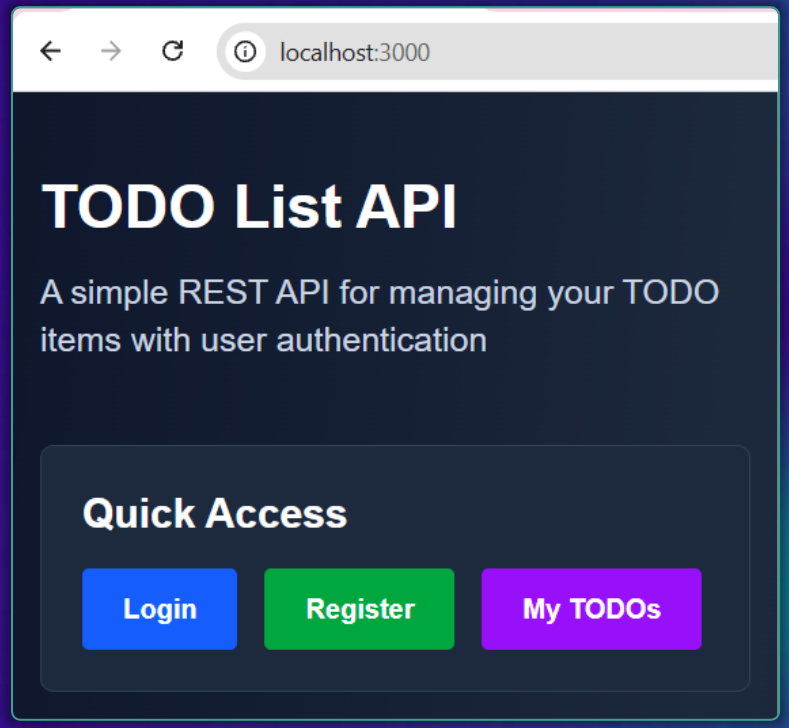
Testing Auth with Postman: Demo

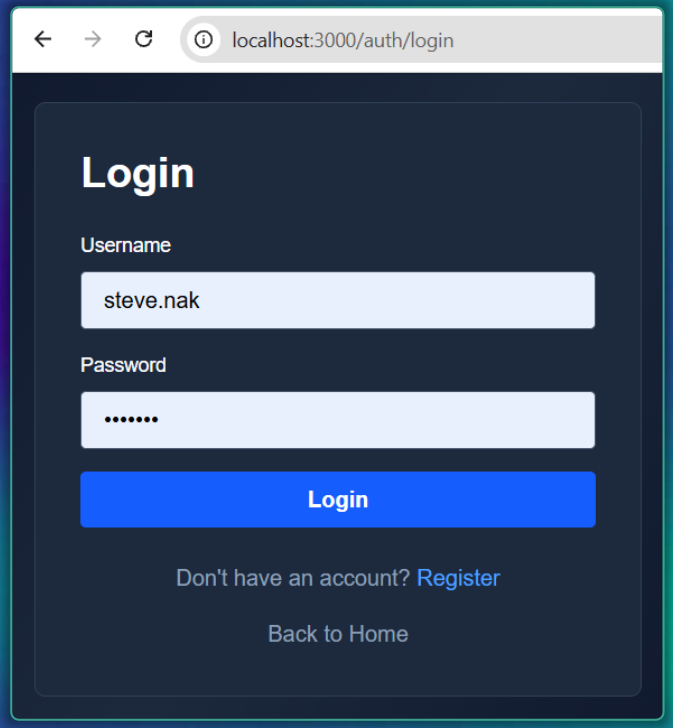
SoftUni AI

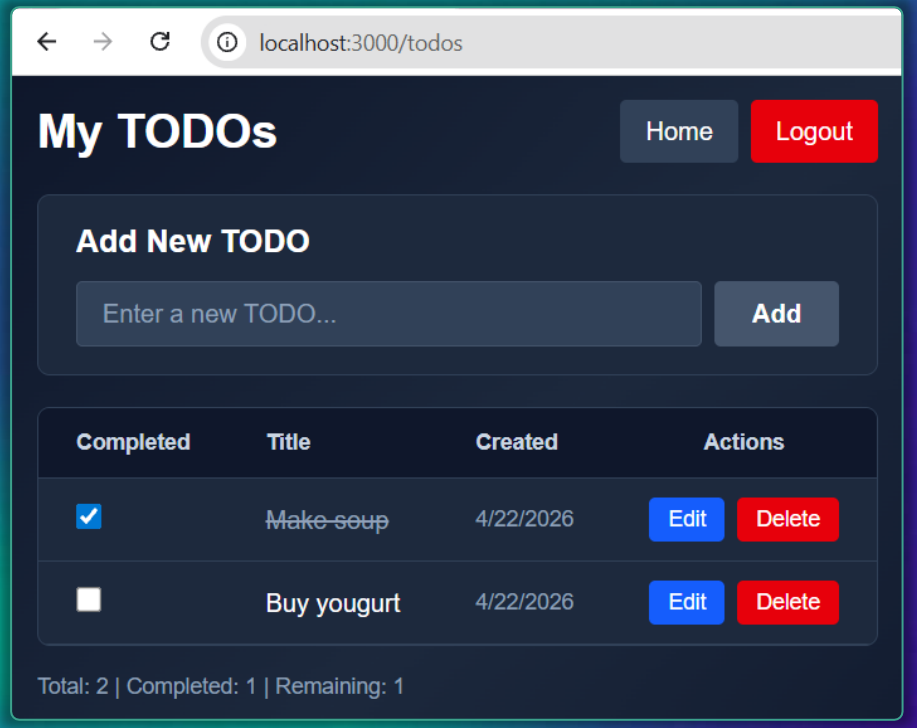


TODO List Client App with Auth

Register / Login UI, Storing Auth Tokens, Protected Routes







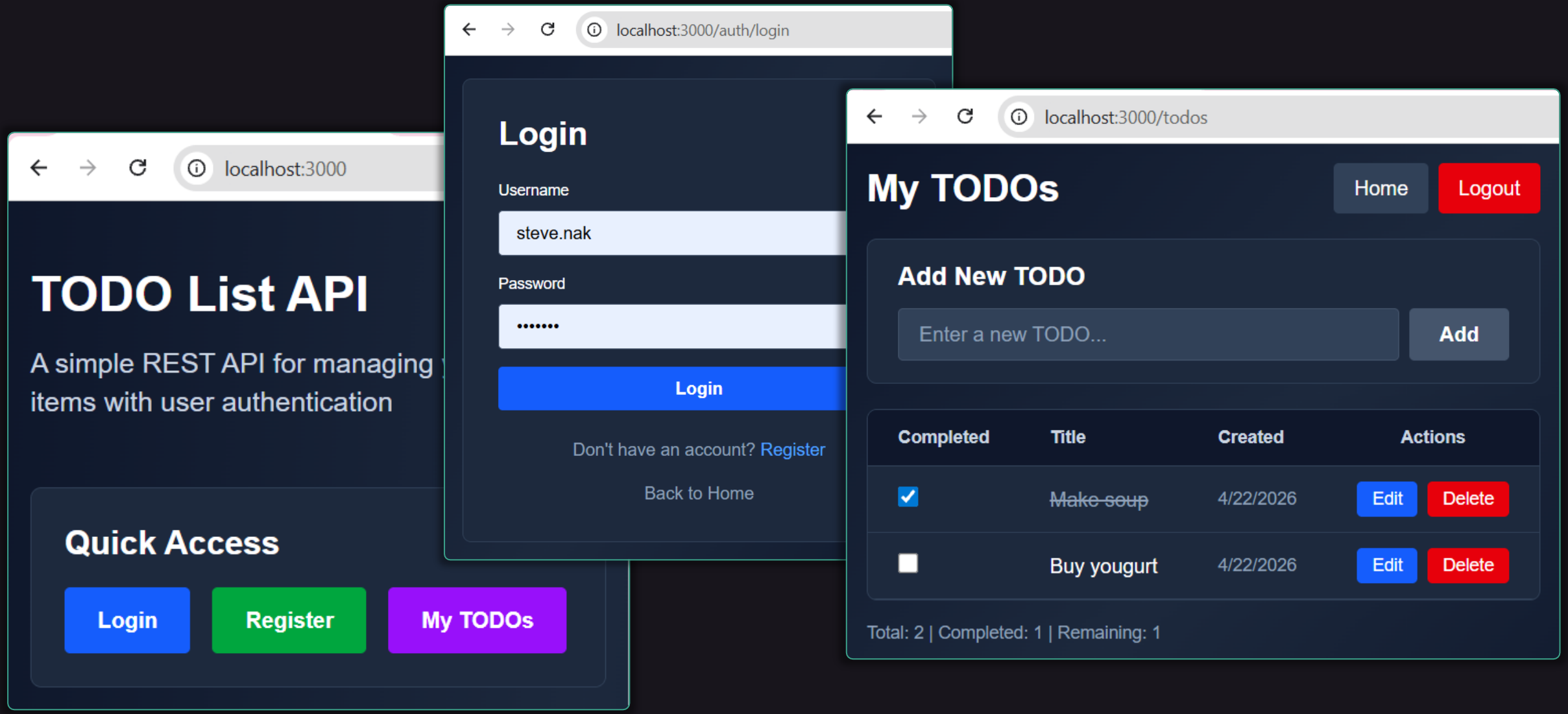
TODO List Client App — AI Prompt

SoftUni AI

- **AI prompt** for **TODO List client app** with **auth** in **Next.js**:

```
Implement a TODO List client app in Next.js:
  - Implement auth: login / register / logout
  - A new page: /todos – in the same Next.js project
    - View the list of TODOs – in a table
    - Toggle `Completed` – checkbox for each row
    - Edit title inline – [Edit] button for each row
    - Delete a TODO – [Delete] button for each row
    - Add a new TODO – input field + [Add] button
  - Use the existing API: fetched from `/api`
  - App Home page: links to auth / TODOs / docs
```

TODO List Client App – Result



The Plan – Client App with Auth

SoftUni AI

📄 Auth Pages

- **/auth/register** — register form
  - Username + password + [Register] button
  - **POST /api/auth/register**
- **/auth/login** — sign in form
  - Username + password + [Login] button
  - **POST /api/auth/login**

🔒 Protected /todos Page

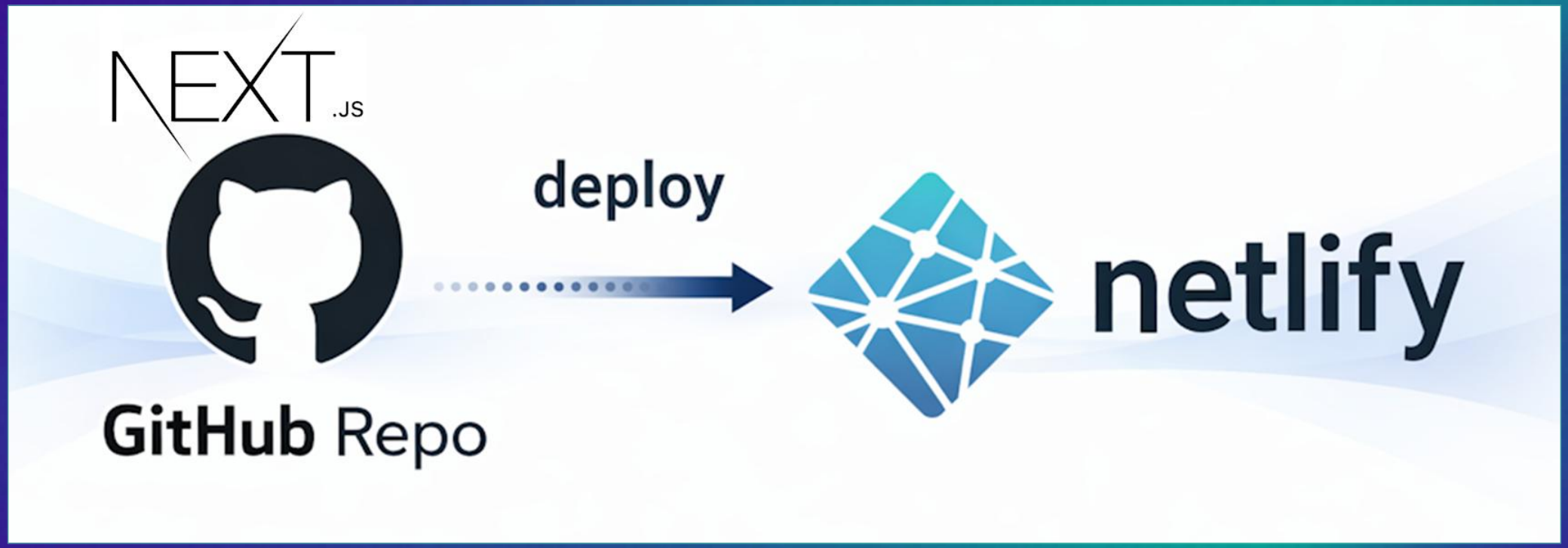
- **Login**: store the **JWT token** in **localStorage**
- On **/todos** mount: check for **token**
  - **Load TODOs** if token is available
  - Redirect to **/auth/login** if missing
- Send Authorization: **Bearer <token>** on API calls
- **Logout**: clear **localStorage** token + go to **/login**
- Top nav: links to **/todos**, **/login**, **/register**, **[Logout]** button

🔒 Security Warning

- **Auth token** in **localStorage** might be attacked with **XSS** (Cross-Site Scripting)

Deploy Next.js API to Netlify

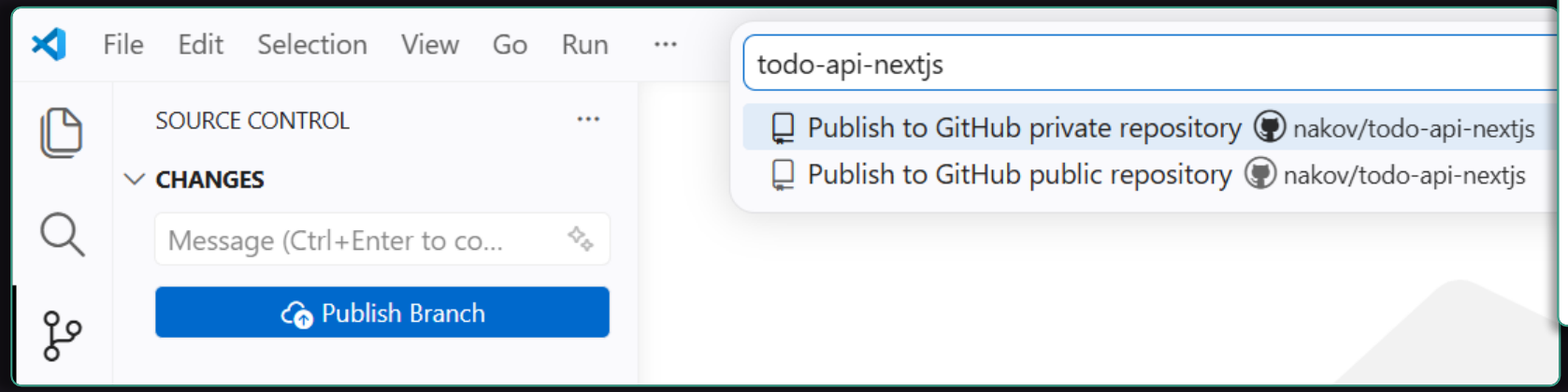
Publish to GitHub, Deploy Next.js App to Netlify, Serverless Limitations

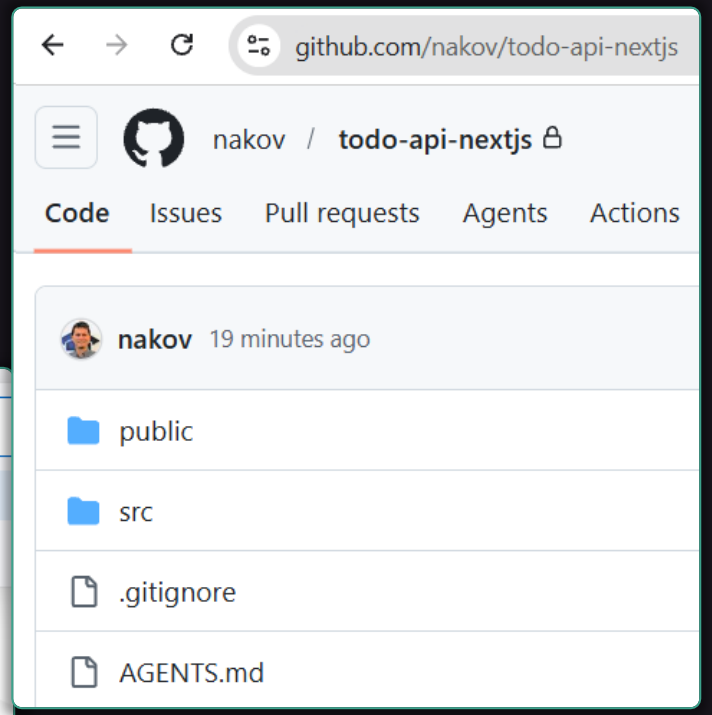


Deploy an Next.js API to Netlify

SoftUni AI

- How to **deploy a Next.js API** project to **Netlify**?
  1. **Publish** your project to **GitHub**
  2. **Deploy to Netlify** from GitHub
- Publish to **GitHub** from **VS Code**



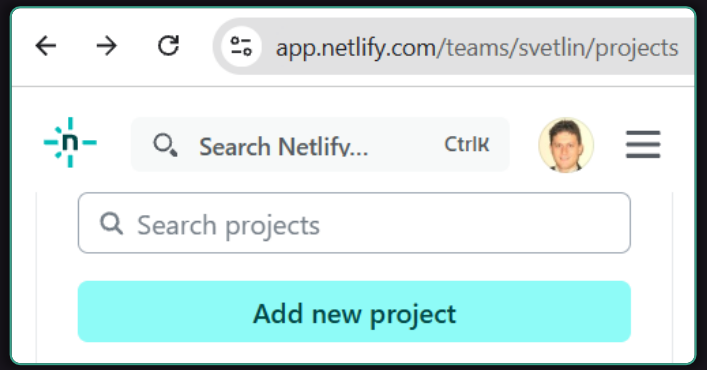


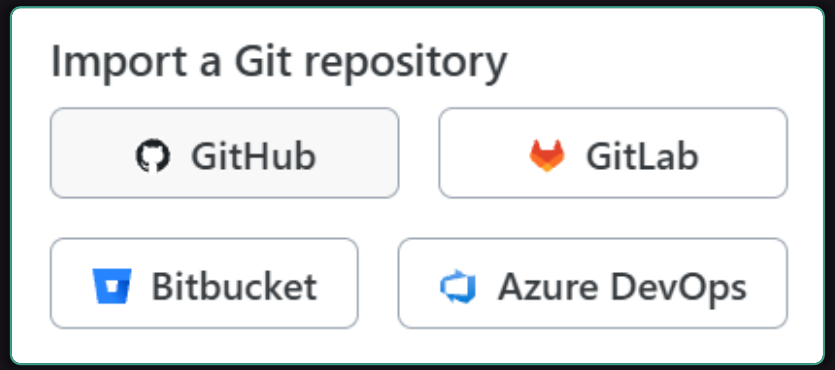
  - By default, `**.gitignore**` will skip `**.env.local**` (config secrets), also folders **node_modules/** and **.next/**

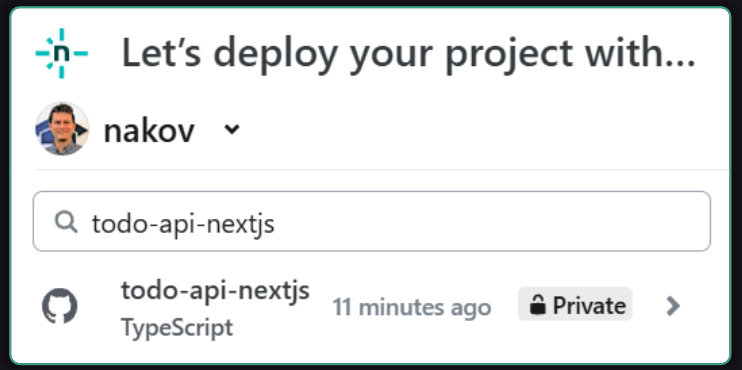
Deploy to Netlify from GitHub

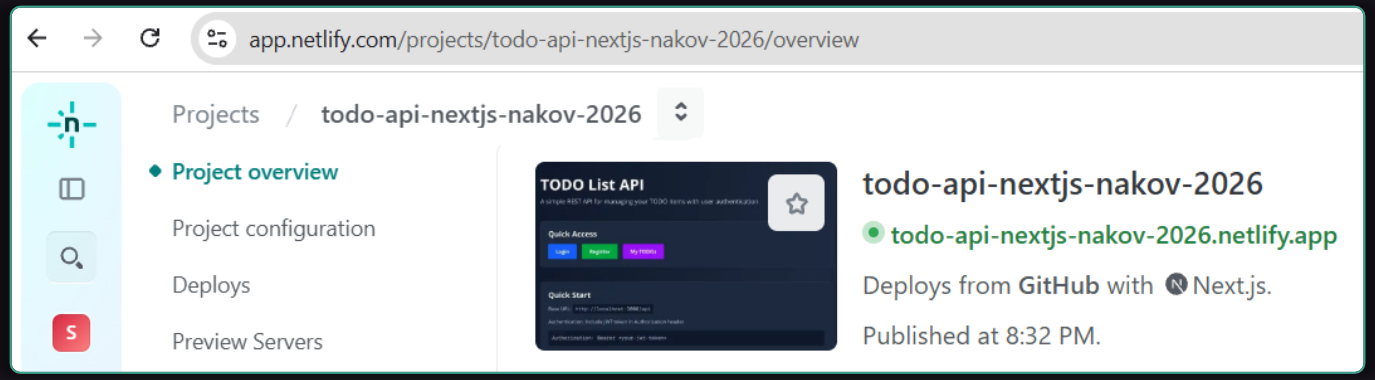
SoftUni AI

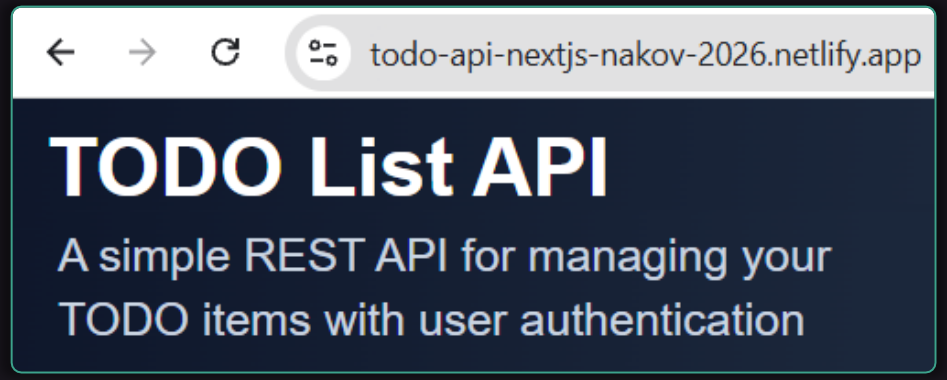
- **Deploy to Netlify** from GitHub
  - Netlify Dashboard → [Add New Project] → [GitHub] → Authorize → Select Project → Configure → Deploy





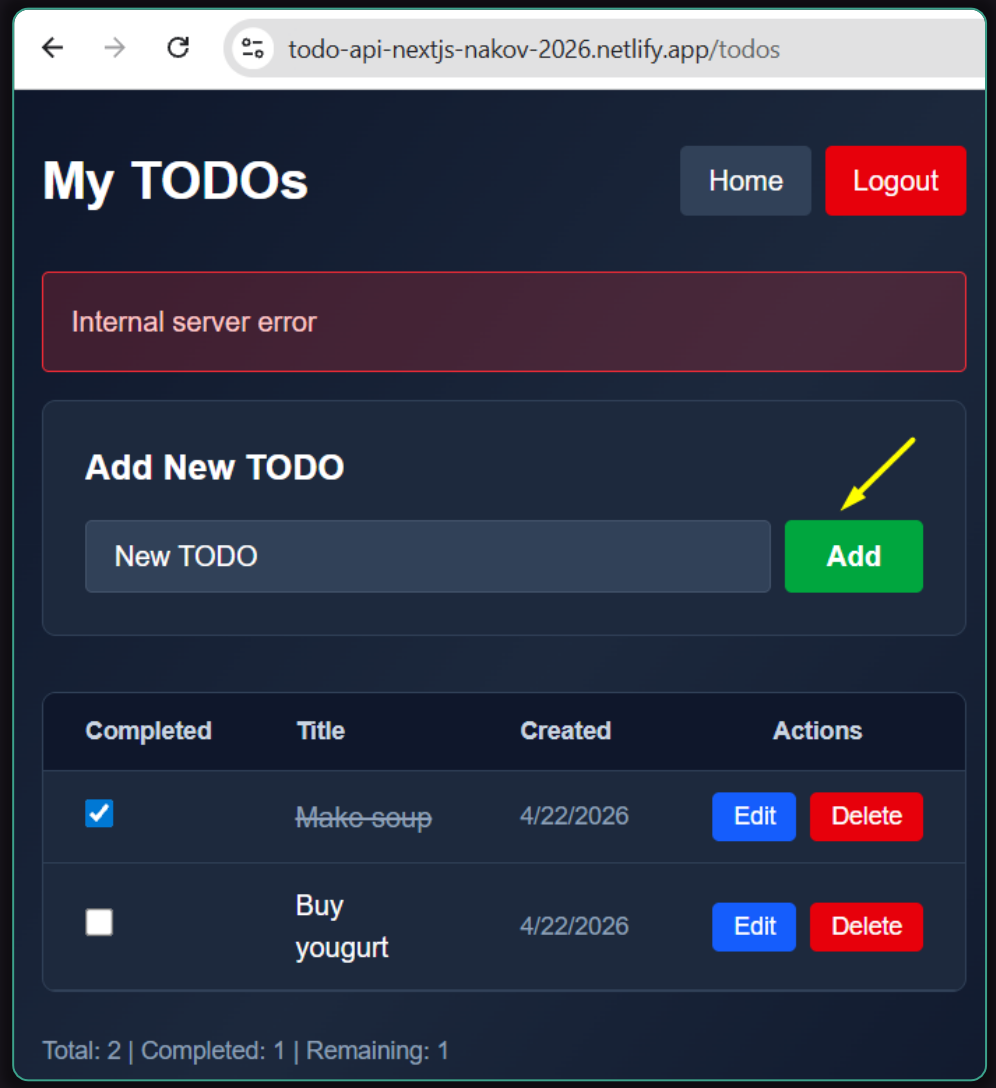


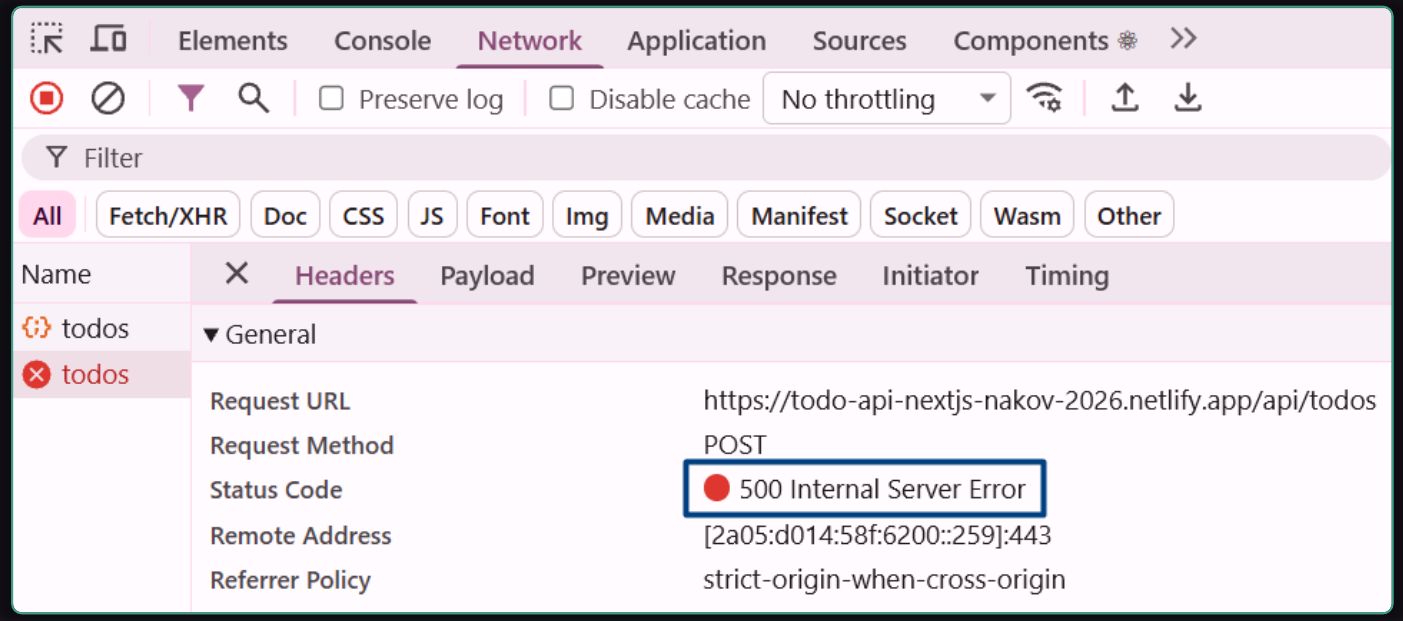




The App Works in Read-Only Mode!

SoftUni AI





- Netlify has **read-only** file system
  - API **read** requests will **work**, but API **write** requests will **fail**

Netlify – A Serverless Environment

- **Netlify** is a **serverless** deployment platform
  - In Netlify app's `**data**` folder in file system is **read-only** (by design)
- How **serverless** platforms work?
  - Requests run on-demand, **without preserving state**
  - **Request** → **server** (from the pool) → run the **code** → return **result**
  - Each request may be served by different runtime (machine)
- How to preserve state?
  - Use a **database** (PostgreSQL), **cache** (Redis), or **external** API
  - Don't persist data in **local files** (even in **/tmp**, which is writable)

How Serverless Actually Runs?

SoftUni AI

⚡ Per-Request Lifecycle

1. A **request** arrives at Netlify's edge network
2. A new **Node.js process** is spawned (or reused from a warm pool)
3. **Route handler runs** → returns a **response**
4. The **process** may be **stopped** shortly after (cold → warm → cold)
   - No guarantee of which **host machine** handles the next request

🚫 Serverless Constraints

- **Memory** does **NOT persist** between requests
- **File system** is **read-only** (our case)
- **/tmp** is writable — but **temporary**, not shared across instances
- No long-running **background tasks**
- Concurrent requests may run on **different host machines**

🖥️ Serverless vs. Server-Based Hosting

- **Server-based** (Railway, Render): long-running process, persistent memory, writable filesystem
- **Serverless** (Netlify, Vercel, AWS Lambda): non-persistent, scales to zero, pay-per-request
- For persistent data in serverless: you need an **external database** — not a local file

What We Learned Today

SoftUni AI

- **Client-Server Model & HTTP** — browsers send HTTP **requests**; servers return status codes and JSON **responses**
- **RESTful API Design** — based on **resources**, addressed by **URL**, using **HTTP** verbs (GET / POST / PATCH / DELETE), status codes and **JSON**
- **Postman** — a powerful developer tool for sending **HTTP requests** and inspecting the **responses**
- **Next.js API Routes Handlers** — building a **RESTful APIs** with TypeScript inside a Next.js project (route handlers in the **/app/api/** folder)
- **JWT Authentication** — server issues **JWT tokens**, encapsulating signed user data; client sends tokens back on each request for authentication
- **Deploying to Netlify** — Next.js projects run natively on Netlify (deploy from GitHub repo) but: **non-persistent** memory, **read-only** file system

Questions?

Postbank – Exclusive Partner for SoftUni AI

- One of the leading **banking institutions** in Bulgaria
- Member of the Eurobank Group with € 103 billion assets
- Innovative trendsetter with next generation **beyond banking**, transforming today, empowering tomorrow
- Certified **Top Employer 2025** by the international Top Employers Institute
- Proven people care and **wellbeing initiatives**
- Benefits and unlimited access to professional, **personal and leadership trainings and programs**
- www.postbank.bg / careers.postbank.bg

Diamond Partners of Software University

Software University

Coca-Cola HBC Bulgaria

Postbank

top EMPLOYER България Bulgaria 2025

FOR A BETTER WORLD OF WORK

SUPER HOSTING .BG

encorp.ai

createX

DRAFT KINGS

THE CROWN IS YOURS

INDEAVR

Serving the high achievers

Diamond Partners of SoftUni Digital

SoftUni Digital

Coca-Cola HBC Bulgaria

Postbank

top EMPLOYER България Bulgaria 2025

FOR A BETTER WORLD OF WORK

SUPER HOSTING .BG

encorp.ai

createX

INDEAVR

Serving the high achievers

DRAFT KINGS

THE CROWN IS YOURS

Diamond Partners of SoftUni Digital

SoftUni Digital

HUMAN

NETPEAK
DIGITAL GROWTH PARTNER

MarmaLAB | E-комерс агенция

ETIENYANEV
Break Your *Limits*. Live Your *Brand*.

1 FOR FIT

Aleikov Studio

Diamond Partners of SoftUni Creative

SoftUni Creative

Coca-Cola HBC Bulgaria

Postbank

top EMPLOYER България Bulgaria 2025

FOR A BETTER WORLD OF WORK

SUPER HOSTING .BG

encorp.ai

createX

INDEAVR

Serving the high achievers

DRAFT KINGS

THE CROWN IS YOURS

Organization Partners of SoftUni Creative

SoftUni Creative

creative corner

Studio π

Think Big • Design Smart

THE BUCKS TOWN'S WORK

DEFFECTO